

داده کاوی

دانشگاه آزاد شیراز - دانشکده مهندسی کامپیوتر

ترم اول سال تحصیلی ۱۴۰۴-۱۴۰۵

فصل شش: Classification: basic concepts and methods

استاد: دکتر امین اسکندری

تیم طراحی محتوای آموزشی و ویراستاری (علمی و ادبی): حمید نامجو و امیرحسین همتی

فهرست

- 6.1 Basic concepts
 - 6.1.1 What is classification?
 - 6.1.2 General approach to classification
- 6.2 Decision tree induction
 - 6.2.1 Decision tree induction
 - 6.2.2 Attribute selection measures
 - 6.2.3 Tree pruning
- 6.3 Bayes classification methods
 - 6.3.1 Bayes' theorem
 - 6.3.2 Naïve Bayesian classification
- 6.4 Lazy learners (or learning from your neighbors)
 - 6.4.1 k-nearest-neighbor classifiers
 - 6.4.2 Case-based reasoning
- 6.5 Linear classifiers
 - 6.5.1 Linear regression
 - 6.5.2 Perceptron: turning linear regression to classification
 - 6.5.3 Logistic regression
- 6.6 Model evaluation and selection
 - 6.6.1 Metrics for evaluating classifier performance
 - 6.6.2 Holdout method and random subsampling
 - 6.6.3 Cross-validation
 - 6.6.4 Bootstrap
 - 6.6.5 Model selection using statistical tests of significance
 - 6.6.6 Comparing classifiers based on cost-benefit and ROC curves
- 6.7 Techniques to improve classification accuracy
 - 6.7.1 Introducing ensemble methods
 - 6.7.2 Bagging
 - 6.7.3 Boosting
 - 6.7.4 Random forests
 - 6.7.5 Improving classification accuracy of class-imbalanced data

Section 6.2

6.2.1. Decision tree induction

القای درخت تصمیم

القای درخت تصمیم یکی از تکنیک‌های پرکاربرد یادگیری ماشین برای **طبقه‌بندی** و **رگرسیون** است. این مدل به شکل یک درخت سلسله‌مراتبی است که در آن هر گره نمایانگر یک **تصمیم بر اساس یک ویژگی** است و هر شاخه نتیجه آن تصمیم را نشان می‌دهد و به گره بعدی منجر می‌شود. برگ‌های درخت نیز خروجی نهایی یا برچسب پیش‌بینی‌شده را ارائه می‌دهند.

🔧 مراحل القای درخت تصمیم

۱. **انتخاب گره ریشه:** الگوریتم ویژگی‌ای را انتخاب می‌کند که بهترین تقسیم‌بندی داده‌ها را ایجاد می‌کند. این انتخاب بر اساس معیارهایی مانند **بهره اطلاعاتی** (Information Gain) یا **ناخالصی گینی** (Gini Impurity) صورت می‌گیرد.

۲. **تقسیم گره‌ها:** داده‌ها بر اساس ویژگی انتخاب‌شده به زیرمجموعه‌هایی تقسیم می‌شوند که هر کدام به یک شاخه جدید منجر می‌روند.

۳. **فرآیند بازگشتی:** مراحل ۱ و ۲ به‌صورت بازگشتی بر روی هر زیرمجموعه اعمال می‌شوند تا زمانی که معیار توقفی (مانند رسیدن به حداکثر عمق درخت یا تعداد کمینه نمونه در یک گره) برقرار شود.

۴. **اختصاص برچسب:** در نهایت، به هر برگ یک برچسب (در طبقه‌بندی) یا مقدار میانگین (در رگرسیون) بر اساس داده‌های موجود در آن گره اختصاص داده می‌شود.

🌸 مثال عملی در پایتون: طبقه‌بندی گل ایریس

بیایید از معروف‌ترین مجموعه داده در یادگیری ماشین — **مجموعه داده Iris** — استفاده کنیم. هدف، طبقه‌بندی گل‌های ایریس به سه گونه *setosa*، *versicolor* و *virginica* بر اساس چهار ویژگی: طول و عرض گلبرگ (petal) و طول و عرض کاسبرگ (sepal) است.

```
In [1]: from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
```

```
# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'datasets' برای بارگیری داده‌های استاندارد
# 'DecisionTreeClassifier' و 'export_text' برای ساخت و تفسیر درخت تصمیم همراه با
```

```
# رو بارگیری می‌کنه **Iris** این بخش، مجموعه داده معروف
# این دیتاست شامل ۱۵۰ نمونه از گل‌ها با ۴ ویژگی (مثل طول و عرض گلبرگ) و ۳ کلاس مختلف (نوع گل) هست
# این داده‌ها برای آموزش الگوریتم‌های یادگیری ماشین و تفسیر مدل بسیار مناسب‌اند
iris = datasets.load_iris()
X = iris.data
y = iris.target
```

```
# این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه
# داده برای آموزش مدل و ۲۰٪ برای ارزیابی عملکردش استفاده می‌شه ۱۰٪
# تضمین می‌کنه که نتایج قابل تکرار باشن `random_state=42`
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# ایجاد می‌شه **Decision Tree** اینجا یک درخت تصمیم
# به پیش‌بینی می‌رسه، (if-else مثل) درخت تصمیم یک الگوریتم یادگیری ماشین قابل تفسیره که با تقسیم فضای ویژگی بر اساس قوانین ساده
clf = DecisionTreeClassifier()
```

```
# می‌کنه (fit) **این بخش، مدل رو روی داده‌های آموزشی** ساخت
# در طی این فرآیند، درخت تصمیم یاد می‌گیره که چطور از ویژگی‌ها برای طبقه‌بندی گل‌ها استفاده کنه
clf.fit(X_train, y_train)
```

```
# اینجا مدل، پیش‌بینی‌ها رو برای داده‌های آزمون انجام می‌ده
# این پیش‌بینی‌ها بعداً با برچسب واقعی مقایسه می‌شن تا دقت مدل سنجیده بشه
predictions = clf.predict(X_test)
```

```
# در اینجا، **دقت مدل** روی داده‌های آزمون محاسبه و چاپ می‌شه
# درصد نمونه‌هایی هست که مدل به درستی طبقه‌بندی کرده (Accuracy) دقت
accuracy = clf.score(X_test, y_test)
print(f"Accuracy: {accuracy:.2f}")

# در نهایت، **قوانین درخت تصمیم** به صورت متنی قابل خواندن تولید می‌شن
# نمایش می‌ده که کاملاً قابل تفسیره if-else ساختار درخت رو به شکل یک سری قانون `export_text`
# این بخش به ما کمک می‌کنه بفهمیم مدل چطور تصمیم می‌گیره و چه ویژگی‌هایی مهمتر هستن
tree_rules = export_text(clf, feature_names=iris.feature_names)
print("Decision Tree Rules:\n", tree_rules)
```

```
Accuracy: 1.00
Decision Tree Rules:
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal length (cm) <= 4.75
|   |   |--- petal width (cm) <= 1.65
|   |   |   |--- class: 1
|   |   |   |--- petal width (cm) > 1.65
|   |   |   |   |--- class: 2
|   |--- petal length (cm) > 4.75
|   |   |--- petal width (cm) <= 1.75
|   |   |   |--- petal length (cm) <= 4.95
|   |   |   |   |--- class: 1
|   |   |   |   |--- petal length (cm) > 4.95
|   |   |   |   |   |--- petal width (cm) <= 1.55
|   |   |   |   |   |   |--- class: 2
|   |   |   |   |   |   |--- petal width (cm) > 1.55
|   |   |   |   |   |   |   |--- petal length (cm) <= 5.45
|   |   |   |   |   |   |   |   |--- class: 1
|   |   |   |   |   |   |   |   |--- petal length (cm) > 5.45
|   |   |   |   |   |   |   |   |   |--- class: 2
|   |   |--- petal width (cm) > 1.75
|   |   |   |--- petal length (cm) <= 4.85
|   |   |   |   |--- sepal length (cm) <= 5.95
|   |   |   |   |   |--- class: 1
|   |   |   |   |   |--- sepal length (cm) > 5.95
|   |   |   |   |   |   |--- class: 2
|   |   |--- petal length (cm) > 4.85
|   |   |   |--- class: 2
```

توضیحات مثال کد پایتون

در این مثال، از کتابخانه **scikit-learn** برای ایجاد یک **DecisionTreeClassifier** استفاده شده است. مدل روی مجموعه داده **Iris** آموزش داده شده، پیش‌بینی‌ها انجام شده و دقت آن محاسبه شده است. همچنین از تابع **export_text** برای نمایش قواعد درخت تصمیم به صورت متنی و قابل فهم توسط انسان استفاده شده است. این خروجی نشان می‌دهد که چگونه مدل بر اساس مقادیر ویژگی‌ها (مانند طول گلبرگ) به سه گونه گل ایریس دسته‌بندی می‌کند. این رویکرد شفافیت و قابلیت تفسیرپذیری بالای درخت تصمیم را به خوبی نشان می‌دهد.

6.2.2. Attribute selection measures

معیارهای انتخاب ویژگی در داده‌کاوی

در داده‌کاوی و یادگیری ماشین، **انتخاب ویژگی** (Attribute Selection) به فرآیند انتخاب زیرمجموعه‌ای از ویژگی‌های مرتبط از بین مجموعه‌ای بزرگ‌تر اشاره دارد. هدف این است که عملکرد مدل بهبود یابد، از **بیش‌برازش** (Overfitting) کاسته شود و مدل قابل‌فهم‌تر و کارآمدتر گردد. معیارهای مختلفی برای ارزیابی اهمیت ویژگی‌ها وجود دارند که هر کدام بر اساس معیارهای خاصی عمل می‌کنند.

مفاهیم کلیدی

1. **بهره اطلاعاتی (Information Gain):** این معیار میزان کاهش بی‌نظمی (آنتروپی) را پس از تقسیم داده‌ها بر اساس یک ویژگی اندازه‌گیری می‌کند. هرچه بهره اطلاعاتی یک ویژگی بیشتر باشد، توانایی آن در جداسازی کلاس‌ها بیشتر است.

2. **نسبت بهره (Gain Ratio):** این معیار از بهره اطلاعاتی استفاده می‌کند اما آن را با توجه به تعداد شاخه‌های ایجادشده توسط

یک ویژگی تعدیل می‌کند. این کار از انتخاب ویژگی‌هایی با تعداد مقادیر زیاد (مثل شناسه) جلوگیری می‌کند.

3. **شاخص جینی (Gini Index):** این معیار میزان ناخالصی یک مجموعه داده را اندازه‌گیری می‌کند. مقدار پایین جینی

نشان‌دهنده خلوص بیشتر و تقسیم‌بندی بهتر است. در درخت‌های تصمیم از آن برای انتخاب بهترین ویژگی برای تقسیم استفاده می‌شود.

4. **کای-اسکوئر (Chi-square):** این آزمون آماری استقلال بین یک ویژگی و کلاس را بررسی می‌کند. ویژگی‌هایی که رابطه قوی‌تری با کلاس دارند، مقدار کای-اسکوئر بالاتری دارند و به عنوان ویژگی‌های معنادار انتخاب می‌شوند.

مثال عملی در پایتون: انتخاب ویژگی با بهره اطلاعاتی

بیایید دوباره از مجموعه داده معروف **Iris** استفاده کنیم و با به‌کارگیری معیار **بهره اطلاعاتی**، مهم‌ترین ویژگی‌ها را برای طبقه‌بندی گونه‌های گل ایریس انتخاب کنیم:

In [2]:

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.feature_selection import SelectKBest, mutual_info_classif

# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'datasets' برای بارگیری داده، 'train_test_split' برای تقسیم داده،
# 'DecisionTreeClassifier' برای ساخت مدل، 'export_text' برای تفسیرپذیری و
# 'SelectKBest'، 'mutual_info_classif' (Information Gain) برای انتخاب ویژگی بر اساس بهره اطلاعاتی

# رو بارگیری می‌کنه **Iris** این بخش، مجموعه داده معروف
# این دیتاست شامل ۱۵۰ نمونه از گل‌ها با ۴ ویژگی (مثل طول و عرض گلبرگ) و ۳ کلاس مختلف (نوع گل) هست
# این داده‌ها برای آموزش مدل‌های یادگیری ماشین و تحلیل انتخاب ویژگی مناسب‌اند
iris = datasets.load_iris()
X = iris.data
y = iris.target

# این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه
# داده برای آموزش و ۲۰٪ برای ارزیابی مدل استفاده می‌شه ۸۰٪
# تضمین می‌کنه که تقسیم‌بندی قابل تکرار باشه `random_state=42`
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# برای انتخاب بهترین ویژگی‌ها استفاده می‌کنه **Mutual Information** این بخش، از بهره اطلاعات متقابل
# ویژگی‌هایی که بیشترین ارتباط اطلاعاتی با برجسب کلاس دارن رو انتخاب می‌کنه `mutual_info_classif` با معیار `SelectKBest`
# ما فقط ۲ ویژگی از ۴ ویژگی اصلی رو نگه می‌داریم تا مدل ساده‌تر و قابل تفسیرتر بشه
selector = SelectKBest(mutual_info_classif, k=2)
X_train_selected = selector.fit_transform(X_train, y_train) # برازش و تبدیل داده آموزشی
X_test_selected = selector.transform(X_test) # تبدیل داده آزمون با همان انتخاب

# ایجاد می‌شه **Decision Tree** اینجا یک درخت تصمیم
# درخت تصمیم یک مدل قابل تفسیر که می‌تونه برای داده‌های انتخاب‌شده آموزش ببینه
clf = DecisionTreeClassifier()

# این بخش، مدل رو روی داده‌های آموزشی که فقط شامل ویژگی‌های انتخاب‌شده هستند، آموزش می‌ده
clf.fit(X_train_selected, y_train)

# اینجا مدل، پیش‌بینی‌ها رو برای داده‌های آزمون انجام می‌ده
# داده‌های آزمون هم قبلاً با همان انتخاب ویژگی تبدیل شدن
predictions = clf.predict(X_test_selected)

# در اینجا، **دقت مدل** روی داده‌های آزمون محاسبه و چاپ می‌شه
# هدف اینه که ببینیم با حذف ویژگی‌های کم‌اهمیت، چقدر از دقت افت می‌کنه یا حفظ می‌شه
accuracy = clf.score(X_test_selected, y_test)
print(f"Accuracy: {accuracy:.2f}")

# در نهایت، **قوانین درخت تصمیم** برای ویژگی‌های انتخاب‌شده به صورت متنی نمایش داده می‌شه
# چون فقط ۲ ویژگی انتخاب شدن، درخت ساده‌تر و تفسیرپذیرتره
# استخراج می‌شه `get_support` نام ویژگی‌های انتخاب‌شده با استفاده از
selected_feature_names = [iris.feature_names[i] for i in selector.get_support(indices=True)]
tree_rules = export_text(clf, feature_names=selected_feature_names)
print("Decision Tree Rules for Selected Features:\n", tree_rules)
```

```
Accuracy: 1.00
Decision Tree Rules for Selected Features:
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal length (cm) <= 4.75
|   |   |--- petal width (cm) <= 1.65
|   |   |--- class: 1
|   |   |--- petal width (cm) > 1.65
|   |   |--- class: 2
|   |--- petal length (cm) > 4.75
|   |   |--- petal width (cm) <= 1.75
|   |   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |   |--- petal length (cm) > 4.95
|   |   |   |   |--- petal width (cm) <= 1.55
|   |   |   |   |--- class: 2
|   |   |   |   |--- petal width (cm) > 1.55
|   |   |   |   |   |--- petal length (cm) <= 5.45
|   |   |   |   |   |--- class: 1
|   |   |   |   |   |--- petal length (cm) > 5.45
|   |   |   |   |   |--- class: 2
|   |   |--- petal width (cm) > 1.75
|   |   |--- petal length (cm) <= 4.85
|   |   |--- class: 2
|   |   |--- petal length (cm) > 4.85
|   |   |--- class: 2
```

توضیحات مثال کد پایتون

در این مثال، از تابع **SelectKBest** کتابخانه **scikit-learn** استفاده شده است که با بهره‌گیری از **اطلاع متقابل (Mutual Information)** به عنوان تابع امتیازدهی، **k** ویژگی برتر با بیشترین بهره اطلاعاتی را انتخاب می‌کند. سپس یک **درخت تصمیم** روی این ویژگی‌های انتخاب‌شده آموزش داده شده و دقت آن ارزیابی می‌شود. این مثال نشان می‌دهد که چگونه استفاده از معیارهای انتخاب ویژگی می‌تواند با حذف ویژگی‌های نامرتب‌ب یا کم‌اهمیت، عملکرد مدل یادگیری ماشین را بهبود بخشد، از بیش‌برازش کاسته و تفسیرپذیری مدل را افزایش دهد.

6.2.3. Tree pruning

هرس درخت

هرس درخت (Tree Pruning) تکنیکی برای جلوگیری از **بیش‌برازش (Overfitting)** در الگوریتم‌های درخت تصمیم است. بیش‌برازش زمانی رخ می‌دهد که درخت بیش از حد پیچیده شود و نه تنها الگوهای داده، بلکه **سر و صدا (Noise)** و نوسانات تصادفی داده‌های آموزشی را نیز یاد بگیرد. این امر باعث عملکرد ضعیف مدل روی داده‌های جدید می‌شود. هرس درخت با حذف بخش‌هایی از درخت که قدرت پیش‌بینی معناداری ندارند، مدل را ساده‌تر و تعمیم‌پذیرتر می‌کند.

انواع هرس درخت

1. **هرس پیش‌رو (Pre-pruning / Early Stopping):** در این روش، فرآیند ساخت درخت از قبل متوقف می‌شود تا از پیچیدگی بیش از حد جلوگیری شود. راهکارهای رایج شامل: - محدود کردن **حداکثر عمق** درخت، - تعیین **حداقل تعداد نمونه‌ها برای تقسیم** یک گره، - یا تعیین **حداقل تعداد نمونه‌ها در یک برگ**.

2. **هرس پس‌رو (Post-pruning / Pruning After Tree Construction):** در این روش، ابتدا درخت کامل ساخته می‌شود و سپس گره‌هایی که بهبود قابل توجهی در دقت پیش‌بینی ایجاد نمی‌کنند، حذف یا ساده‌سازی می‌شوند. یکی از روش‌های معروف،

هرس پیچیدگی-هزینه (Cost-Complexity Pruning) است که با استفاده از یک پارامتر تنظیم‌کننده (**alpha**) تعادلی بین دقت مدل و پیچیدگی درخت ایجاد می‌کند.

مثال عملی در پایتون: هرس پس‌رو با روش پیچیدگی-هزینه

بیا یاد دوباره از مجموعه داده Iris استفاده کنیم، یک درخت تصمیم کامل بسازیم و سپس از تکنیک هرس پیچیدگی-هزینه (Cost-Complexity Pruning) برای ساده‌سازی درخت و جلوگیری از بیش‌برازش استفاده کنیم:

```
In [3]: from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.model_selection import GridSearchCV

# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'datasets' داده 'train_test_split'، برای بارگیری داده
# 'DecisionTreeClassifier' برای ساخت مدل 'export_text' برای تفسیر درخت و
# 'GridSearchCV' برای بهینه‌سازی هایپرپارامترها

# رو بارگیری می‌کنه **Iris** این بخش، مجموعه داده معروف
# این دیتاست شامل ۱۵۰ نمونه از گل‌ها با ۴ ویژگی (مثل طول و عرض گلبرگ) و ۳ کلاس مختلف (نوع گل) هست
# این داده‌ها برای آموزش و ارزیابی مدل‌های یادگیری ماشین بسیار مناسب‌اند
iris = datasets.load_iris()
X = iris.data
y = iris.target

# این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه
# داده برای آموزش و ۲۰٪ برای ارزیابی مدل استفاده می‌شه ۸۰٪
# تضمین می‌کنه که تقسیم‌بندی قابل تکرار باشه `random_state=42`
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ایجاد می‌شه ** (Decision Tree) اینجا یک درخت تصمیم
# داره (overfitting) درخت تصمیم یک مدل قابل تفسیره، اما تمایل به اضافه‌برازش
# استفاده می‌کنیم ** (Cost-Complexity Pruning) برای جلوگیری از این مشکل، از **کاهش پیچیدگی با هزینه
clf = DecisionTreeClassifier()

# تعریف می‌کنه `ccp_alpha` این بخش، شبکه‌ای از مقادیر ممکن برای پارامتر
# هست: هرچه مقدارش بیشتر باشه، درخت ساده‌تر و کوتاه‌تر می‌شه pruning پارامتر تنظیم‌کننده در روش `ccp_alpha`
# اینجا چند مقدار کوچک برای امتحان کردن انتخاب شدن
param_grid = {'ccp_alpha': [0.001, 0.002, 0.003, 0.004, 0.005]}

# رو پیدا کنه `ccp_alpha` استفاده می‌شه تا بهترین مقدار **GridSearchCV** اینجا از
# یعنی اعتبارسنجی متقابل ۵-تایی انجام می‌شه تا عملکرد مدل به‌صورت میانگین روی ۵ بخش ارزیابی بشه `cv=5`
# این روش به ما کمک می‌کنه مدلی انتخاب کنیم که به خوبی تعمیم بده
grid_search = GridSearchCV(clf, param_grid, cv=5)
grid_search.fit(X_train, y_train)

# استخراج می‌شه (`ccp_alpha` با بهترین) پس از جست‌وجوی شبکه‌ای، بهترین مدل
best_clf = grid_search.best_estimator_

# اینجا مدل بهینه‌شده، پیش‌بینی‌ها رو برای داده‌های آزمون انجام می‌ده
predictions = best_clf.predict(X_test)

# در اینجا، **دقت مدل** روی داده‌های آزمون محاسبه و چاپ می‌شه
# چقدر می‌تونه عملکرد تعمیم مدل رو بهبود بده pruning هدف اینه که ببینیم
accuracy = best_clf.score(X_test, y_test)
print(f"Accuracy: {accuracy:.2f}")

# به صورت متنی نمایش داده می‌شه ** (Pruned Tree) در نهایت، **قوانین درخت تصمیم اصلاح‌شده
# این درخت معمولاً کوتاه‌تر و قابل فهم‌تر از درخت کامل هست و از قوانین بی‌معنی جلوگیری می‌کنه
tree_rules = export_text(best_clf, feature_names=iris.feature_names)
print("Decision Tree Rules for Pruned Tree:\n", tree_rules)
```

```
Accuracy: 1.00
Decision Tree Rules for Pruned Tree:
|--- petal width (cm) <= 0.80
|   |--- class: 0
|--- petal width (cm) > 0.80
|   |--- petal length (cm) <= 4.75
|   |   |--- petal width (cm) <= 1.65
|   |   |   |--- class: 1
|   |   |   |--- petal width (cm) > 1.65
|   |   |   |   |--- class: 2
|   |--- petal length (cm) > 4.75
|   |   |--- petal width (cm) <= 1.75
|   |   |   |--- petal length (cm) <= 4.95
|   |   |   |   |--- class: 1
|   |   |   |   |--- petal length (cm) > 4.95
|   |   |   |   |   |--- petal width (cm) <= 1.55
|   |   |   |   |   |   |--- class: 2
|   |   |   |   |   |   |--- petal width (cm) > 1.55
|   |   |   |   |   |   |   |--- sepal length (cm) <= 6.95
|   |   |   |   |   |   |   |   |--- class: 1
|   |   |   |   |   |   |   |   |--- sepal length (cm) > 6.95
|   |   |   |   |   |   |   |   |   |--- class: 2
|   |   |--- petal width (cm) > 1.75
|   |   |   |--- petal length (cm) <= 4.85
|   |   |   |   |--- sepal length (cm) <= 5.95
|   |   |   |   |   |--- class: 1
|   |   |   |   |   |--- sepal length (cm) > 5.95
|   |   |   |   |   |   |--- class: 2
|   |   |--- petal length (cm) > 4.85
|   |   |   |--- class: 2
```

توضیحات مثال کد پایتون

در این مثال، از **GridSearchCV** برای جست‌وجوی بهترین مقدار هایپرپارامتر **ccp_alpha** (مربوط به هرس پیچیدگی-هزینه) استفاده شده است. این پارامتر تعادلی بین **دقت مدل روی داده‌های آموزشی و پیچیدگی درخت** ایجاد می‌کند: هرچه مقدار آن بیشتر باشد، درخت ساده‌تر (کوتاه‌تر) می‌شود. با ارزیابی عملکرد مدل در مقادیر مختلف alpha، مدلی انتخاب می‌شود که بهترین تعادل بین بیش‌برازش و کارایی را داشته باشد. این رویکرد نشان می‌دهد که چگونه هرس پس‌رو می‌تواند منجر به یک درخت تصمیم ساده‌تر، قابل‌تفسیرتر و تعمیم‌پذیرتر شود.

Section 6.3

6.3.1. Bayes’ theorem

قضیه بیز: به‌روزرسانی باورها با شواهد جدید

قضیه بیز (Bayes' Theorem) یکی از مفاهیم بنیادین در نظریه احتمالات است که به نام **توماس بیز** (Reverend Thomas Bayes) نام‌گذاری شده است. این قضیه روشی برای به‌روزرسانی باورها درباره یک فرضیه با توجه به شواهد جدید فراهم می‌کند. فرمول ریاضی آن به صورت زیر است:

$$P(A|B) = [P(B|A) \times P(A)] / P(B)$$

که در آن:

P(A|B): احتمال وقوع پیشامد A به شرط وقوع پیشامد B (احتمال پس از دیدن شواهد). **P(B|A)**: احتمال مشاهده شواهد B به شرط درست بودن فرض A (درست‌نمایی). **P(A)**: احتمال قبلی (Prior) فرض A – یعنی باور اولیه قبل از دیدن شواهد. **P(B)**: احتمال مشاهده شواهد B از همه جهات (عامل نرمال‌سازی).

این قضیه در آمار، یادگیری ماشین و هوش مصنوعی کاربردهای گسترده‌ای دارد، از جمله **طبقه‌بندی ایمیل‌های اسپم، تشخیص پزشکی و سیستم‌های توصیه‌گر**.

مثال عملی در پایتون: تشخیص ایمیل اسپم با پیز

بیایید از معروف‌ترین دیتاست در تشخیص اسپم – **Spambase** – استفاده کنیم. این مجموعه داده شامل ویژگی‌هایی است که بر اساس فراوانی کلمات خاص (مانند "win"، "free") و کاراکترهای خاص (مانند علامت تعجب) در ایمیل‌ها استخراج شده‌اند، همراه با برچسبی که مشخص می‌کند آیا ایمیل اسپم است یا نه. از الگوریتم **نایو پیز** (Naive Bayes) برای آموزش یک مدل طبقه‌بندی استفاده می‌کنیم:

```
In [ ]: # استفاده میکنیم UCI Irvine بجای آپلود دستی فایل دیتاست از لایبری دانشگاه #
pip install ucimlrepo
```

```
In [11]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report
from ucimlrepo import fetch_ucirepo

# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'pandas' برای کار با داده‌ها
# 'train_test_split' برای تقسیم داده‌ها به آموزش و تست
# 'MultinomialNB' برای ساخت مدل نایو پیز
# 'accuracy_score' و 'classification_report' برای ارزیابی عملکرد مدل
# UCI برای بارگیری دیتاست مستقیماً از سایت 'fetch_ucirepo' و در نهایت

# رو بارگیری می‌کنه **Spambase** این بخش، مجموعه داده
# این دیتاست شامل ایمیل‌هایی هست که ویژگی‌های متنی‌شون به صورت عددی کد شده
# هر سطر = یک ایمیل، هر ستون (به جز آخرین) = یک ویژگی عددی (مثل فراوانی یک کلمه یا کاراکتر خاص)
# هست: 0 برای ایمیل معمولی و 1 برای اسپم (label) ستون آخر برچسب
spambase = fetch_ucirepo(id=94)

# رو از هم جدا می‌کنیم (y) و برچسب‌ها (X) اینجا داده‌ها
# features و targets همون ستون‌های عددی هستن
X = spambase.data.features
y = spambase.data.targets.squeeze() # به آرایه یک‌بعدی y تبدیل

# این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه
# داده برای آموزش مدل و ۳۰٪ داده برای تست استفاده می‌شه ۸۰٪
# باعث میشه تقسیم داده‌ها همیشه ثابت بمونه (قابل تکرار) random_state=42
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ساخته می‌شه ** (Multinomial Naive Bayes) اینجا یک طبقه‌بند نایو پیز چندجمله‌ای
# این مدل برای داده‌های گسسته (مثل شمارش کلمات یا فرکانس ویژگی‌ها) مناسب‌تره
# هم ویژگی‌ها شکلی شبیه به شمارش دارن، پس انتخاب مناسبیه Spambase در دیتاست
clf = MultinomialNB()

# این بخش، مدل رو روی داده‌های آموزشی **آموزش** می‌ده
# یاد می‌گیره که y_train و X_train در این مرحله، مدل با استفاده از داده‌های
# الگوهای مربوط به ایمیل اسپم و معمولی رو تشخیص بده
clf.fit(X_train, y_train)

# انجام می‌ده (X_test) اینجا مدل، پیش‌بینی‌ها رو برای داده‌های آزمون
# یعنی برای ایمیل‌هایی که مدل ندیده، برچسب (اسپم یا معمولی) حدس زده می‌شه
predictions = clf.predict(X_test)

# در اینجا، عملکرد مدل ارزیابی می‌شه
# - **Accuracy** (دقت کلی)
# - **Classification Report** (گزارش طبقه‌بندی) شامل precision، recall و F1-score (۱ و ۰)
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test, predictions))
```

Accuracy: 0.79

Classification Report:

	precision	recall	f1-score	support
0	0.80	0.84	0.82	531
1	0.76	0.72	0.74	390
accuracy			0.79	921
macro avg	0.78	0.78	0.78	921
weighted avg	0.79	0.79	0.79	921



توضیحات مثال کد پایتون

در این مثال، از الگوریتم **نایو پیز** (به‌ویژه **Multinomial Naive Bayes**) برای طبقه‌بندی ایمیل‌ها به دو دسته **اسپم** و **غیراسپم** استفاده شده است. این الگوریتم احتمال هر دسته را با استفاده از قضیه پیز و با فرض استقلال ویژگی‌ها (کلمات) محاسبه می‌کند. از **CountVectorizer** برای تبدیل متن ایمیل‌ها به یک ماتریس عددی (شمارش کلمات) استفاده شده است که ورودی

مناسبی برای الگوریتم نایو بیِز محسوب می‌شود. این مثال نشان می‌دهد که چگونه یک مدل ساده اما قدرتمند می‌تواند با استفاده از احتمالات، الگوهای کلمات را یاد بگیرد و ایمیل‌های جدید را با دقت بالا طبقه‌بندی کند.

6.3.2. Naïve Bayesian classification

طبقه‌بندی بیِز: قدرت سادگی

طبقه‌بندی بیِز (Naïve Bayesian Classification) یک تکنیک احتمالاتی بر اساس **قضیه بیِز** است که با فرض «ساده» **استقلال ویژگی‌ها** کار می‌کند. با وجود سادگی و فرض غیرواقعی استقلال، این طبقه‌بندی‌کننده‌ها اغلب در عمل عملکرد خوبی دارند، به‌ویژه در کاربردهایی مانند **طبقه‌بندی متن و تشخیص ایمیل اسپم**.

فرض «ساده» به این معناست که حضور (یا عدم حضور) یک ویژگی خاص در یک کلاس، مستقل از حضور یا عدم حضور سایر ویژگی‌ها است. این فرض محاسبات را بسیار ساده و کارآمد می‌کند، حتی برای مجموعه‌داده‌هایی با تعداد زیادی ویژگی.

فرمول پایه طبقه‌بندی بیِز ساده به صورت زیر است:

$$P(y|X) = [P(X|y) \times P(y)] / P(X)$$

که در آن:

$P(y|X)$: احتمال عضویت در کلاس y با توجه به ویژگی‌های X (احتمال پسین). **$P(X|y)$** : احتمال مشاهده ویژگی‌های X به شرط عضویت در کلاس y . **$P(y)$** : احتمال قبلی کلاس y (بدون داشتن اطلاعات از X). **$P(X)$** : احتمال مشاهده ترکیب ویژگی‌های X (معمولاً به عنوان عامل نرمال‌سازی در نظر گرفته می‌شود).

این مدل به دلیل سرعت بالا، نیاز کم به داده آموزشی و عملکرد قوی در داده‌های متنی، یکی از انتخاب‌های اولیه در پیش‌پردازش و طبقه‌بندی سریع است.

مثال عملی در پایتون: طبقه‌بندی متن با بیِز ساده

بیایید از مجموعه داده معروف **Newsgroups 20** استفاده کنیم که شامل اسناد گروه‌های خبری در ۲۰ موضوع مختلف (مانند ورزش، سیاست، علوم) است. با استفاده از الگوریتم **بیِز ساده** و بردارسازی کلمات، مدلی آموزش می‌دهیم تا موضوع هر سند را بر اساس متن آن تشخیص دهد:

```
In [12]: from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
```

```
# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'fetch_20newsgroups' برای بارگیری داده متنی
# 'MultinomialNB' برای طبقه‌بندی
# 'train_test_split' برای سنجش عملکرد
# 'TfidfVectorizer' برای بارگیری داده متنی

# بارگیری می‌کنه **20newsgroups** این بخش، مجموعه داده **20**
# این دیتاست شامل حدود ۱۸ هزار ایمیل یا پست متنی در ۲۰ موضوع مختلف (مثل علوم، سیاست، ورزش) هست
# هم داده‌های آموزشی و هم آزمون رو بارگیری می‌کنیم `subset='all'` با استفاده از
# باعث می‌شه بخش‌های غیرمحتوایی متن حذف بشن تا مدل روی محتوای واقعی یاد بگیره `remove=('headers', 'footers', 'quotes')`
newsgroups = fetch_20newsgroups(subset='all', remove=('headers', 'footers', 'quotes'))
```

```
# این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه
# داده برای آموزش و ۲۰٪ برای آزمون استفاده می‌شه ۸۰٪
# تضمین می‌کنه که تقسیم‌بندی قابل تکرار باشه `random_state=42`
X_train, X_test, y_train, y_test = train_test_split(newsgroups.data, newsgroups.target, test_size=0.2, random_state=42)

# تبدیل می‌کنه **TF-IDF (Term Frequency-Inverse Document Frequency)** این بخش، متن‌ها رو به فضای عددی با استفاده از روش
# محاسبه می‌شه (IDF) و هم کمیابی کلمه در کل مجموعه داده (TF) وزنی به هر کلمه می‌ده که هم بر اساس فراوانی کلمه در متن TF-IDF
# این روش به مدل کمک می‌کنه کلمات مهم و توصیفی رو از کلمات رایج و بی‌معنا تفکیک کنه
vectorizer = TfidfVectorizer()
X_train_tfidf = vectorizer.fit_transform(X_train) # برازش و تبدیل داده آموزشی
X_test_tfidf = vectorizer.transform(X_test) # تبدیل داده آزمون با همان واژگان آموزشی

# ایجاد می‌شه **Multinomial Naive Bayes) اینجا یک طبقه‌بند نایو بیز چندجمله‌ای
# این مدل برای مسائل طبقه‌بندی متن بسیار مناسب‌ست چون با داده‌های شمارشی (مثل فراوانی کلمات) کار می‌کنه
# فرض استقلال ویژگی‌ها در اینجا به خوبی جواب می‌ده، حتی اگر در عمل کاملاً درست نباشه (این همون "نایو" بودن مدل هست)
clf = MultinomialNB()

# تبدیل شدن، آموزش می‌ده TF-IDF این بخش، مدل رو روی داده‌های آموزشی که با
# مدل یاد می‌گیره که چطور ترکیب کلمات با احتمال تعلق به یک موضوع خاص مرتبط هست
clf.fit(X_train_tfidf, y_train)

# اینجا مدل، پیش‌بینی‌ها رو برای داده‌های آزمون انجام می‌ده
# آماده شدن TF-IDF داده‌های آزمون قبلاً با همان تبدیل
predictions = clf.predict(X_test_tfidf)

# در اینجا، عملکرد مدل با دو معیار ارزیابی می‌شه:
# درصد پیش‌بینی‌های درست: **Accuracy (دقت) -
# برای هر یک از ۲۰ موضوع F1-score شامل دقت، یادآوری و **Classification Report) گزارش طبقه‌بندی -
# باعث می‌شه نام واقعی موضوعات در خروجی نمایش داده بشه `target_names=newsgroups.target_names`
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test, predictions, target_names=newsgroups.target_names))
```

Accuracy: 0.66

Classification Report:

	precision	recall	f1-score	support
alt.atheism	0.86	0.12	0.21	151
comp.graphics	0.70	0.61	0.66	202
comp.os.ms-windows.misc	0.67	0.61	0.64	195
comp.sys.ibm.pc.hardware	0.52	0.78	0.62	183
comp.sys.mac.hardware	0.89	0.64	0.74	205
comp.windows.x	0.89	0.81	0.85	215
misc.forsale	0.86	0.60	0.71	193
rec.autos	0.85	0.73	0.79	196
rec.motorcycles	0.51	0.74	0.61	168
rec.sport.baseball	0.96	0.77	0.86	211
rec.sport.hockey	0.88	0.88	0.88	198
sci.crypt	0.63	0.83	0.71	201
sci.electronics	0.85	0.55	0.67	202
sci.med	0.88	0.69	0.77	194
sci.space	0.82	0.74	0.78	189
soc.religion.christian	0.26	0.96	0.41	202
talk.politics.guns	0.76	0.76	0.76	188
talk.politics.mideast	0.79	0.77	0.78	182
talk.politics.misc	0.95	0.13	0.23	159
talk.religion.misc	1.00	0.01	0.01	136
accuracy			0.66	3770
macro avg	0.78	0.64	0.63	3770
weighted avg	0.77	0.66	0.65	3770



توضیحات مثال کد پایتون

در این مثال، از طبقه‌بند بیز ساده چندجمله‌ای (Multinomial Naive Bayes) روی داده‌های متنی استفاده شده که با روش **TF-IDF** (فراوانی جمله‌ای-معکوس فراوانی سند) به فضای عددی تبدیل شده‌اند. این روش وزن‌دهی، کلماتی که در یک سند زیاد ظاهر می‌شوند اما در کل مجموعه داده نادر هستند را برجسته می‌کند. مدل با یادگیری احتمال وقوع کلمات در هر موضوع، قادر است اسناد جدید را با دقت قابل‌قبولی به یکی از ۲۰ موضوع تخصیص دهد. این مثال نشان می‌دهد که چگونه ترکیب **TF-IDF** و **بیز ساده** یک راه‌حل سریع، کارآمد و موثر برای طبقه‌بندی متن است.

Section 6.4

6.4.1. k-nearest-neighbor classifiers



الگوریتم k-نزدیک‌ترین همسایه (k-NN)

الگوریتم **k-نزدیک‌ترین همسایه** (k-Nearest Neighbor) یک تکنیک ساده و شهودی برای طبقه‌بندی است که بر این ایده استوار است: «نمونه‌های مشابه، احتمالاً به یک دسته تعلق دارند». به عبارت دیگر، این الگوریتم یک نقطه داده را بر اساس اکثریت

کلاسی k نمونه‌ی نزدیک به آن در فضای ویژگی‌ها طبقه‌بندی می‌کند. برای سنجش شباهت بین نمونه‌ها، از معیارهای فاصله مانند فاصله اقلیدسی استفاده می‌شود.

🔧 مراحل کلیدی الگوریتم k -NN

1. **انتخاب مقدار k** : تعداد همسایه‌هایی که برای تصمیم‌گیری در نظر گرفته می‌شوند (مثلاً $k=3$ یا $k=5$). انتخاب مناسب k تعادلی بین حساسیت به نویز و تعمیم‌پذیری ایجاد می‌کند.

2. **محاسبه فواصل**: فاصله بین نمونه هدف و تمام نمونه‌های موجود در مجموعه آموزشی محاسبه می‌شود. رایج‌ترین معیار، فاصله اقلیدسی است، اما فواصل مانهتن یا مینکوفسکی نیز استفاده می‌شوند.

3. **یافتن k نزدیک‌ترین همسایه**: k نمونه‌ای که کمترین فاصله را با نمونه هدف دارند، انتخاب می‌شوند.

4. **رای اکثریت**: برچسب کلاس نمونه هدف بر اساس کلاس غالب بین k همسایه نزدیک تعیین می‌شود. در مسائل رگرسیون، معمولاً میانگین مقدار همسایه‌ها محاسبه می‌شود.

🌸 مثال عملی در پایتون: طبقه‌بندی گل ایریس با k -NN

بیا یاد از معروف‌ترین مجموعه داده در یادگیری ماشین — **مجموعه داده Iris** — استفاده کنیم و با به‌کارگیری الگوریتم **k -NN**، گونه گل‌های ایریس را بر اساس ویژگی‌های گلبرگ و کاسبرگ طبقه‌بندی کنیم:

In [13]:

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'datasets' برای بارگیری داده، 'train_test_split' برای تقسیم داده
# 'KNeighborsClassifier' برای ساخت مدل  $k$ -NN، سنجش عملکرد
iris = datasets.load_iris()
X = iris.data
y = iris.target

# این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه
# داده برای آموزش و ۲۰٪ برای ارزیابی مدل استفاده می‌شه ۸۰٪
# تضمین می‌کنه که تقسیم‌بندی قابل تکرار باشه `random_state=42`
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ایجاد می‌شه **k-Nearest Neighbors (k-NN)** نزدیک‌ترین همسایه  $k$  اینجا یک طبقه‌بند
# نمونه نزدیکش در داده‌های آموزشی نسبت می‌ده  $k$  این مدل یک الگوریتم غیرپارامتریک ساده‌ست که بر اساس شباهت (مثلاً فاصله اقلیدسی) نمونه جدید رو به
# انتخاب شده، یعنی تصمیم‌گیری بر اساس ۳ نمونه نزدیک‌تر صورت می‌گیره  $k=3$  در اینجا
clf = KNeighborsClassifier(n_neighbors=3)

# این بخش، مدل رو روی داده‌های آموزشی **آموزش** می‌ده
# آموزش "واقعی" نداره، بلکه تمام داده‌های آموزشی رو ذخیره می‌کنه تا در زمان پیش‌بینی ازشون استفاده کنه "  $k$ -NN"، در واقع
clf.fit(X_train, y_train)

# اینجا مدل، پیش‌بینی‌ها رو برای داده‌های آزمون انجام می‌ده
# نزدیک‌ترین همسایه در داده‌های آموزشی پیدا می‌شه و اکثریت کلاس‌ها تعیین‌کننده پیش‌بینی هست  $k$ ، برای هر نمونه آزمون
predictions = clf.predict(X_test)

# در اینجا، عملکرد مدل با دو معیار ارزیابی می‌شه:
# درصد پیش‌بینی‌های درست **Accuracy** (دقت) -
# برای هر کلاس  $F1$ -score شامل دقت، یادآوری و **Classification Report** گزارش طبقه‌بندی -
# در خروجی نمایش داده بشه ('setosa' مثل) باعث می‌شه نام واقعی گل‌ها `target_names=iris.target_names`
accuracy = accuracy_score(y_test, predictions)
```

```
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test, predictions, target_names=iris.target_names))
```

Accuracy: 1.00

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	10
versicolor	1.00	1.00	1.00	9
virginica	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

توضیحات مثال کد پایتون

در این مثال، از کتابخانه **scikit-learn** برای ایجاد یک طبقه‌بند **k-نزدیک‌ترین همسایه** با **k=3** استفاده شده است. مدل روی مجموعه داده **Iris** آموزش داده شده و پیش‌بینی‌ها بر روی داده‌های آزمون انجام شده است. سپس دقت کلی مدل و گزارش طبقه‌بندی (شامل دقت، یادآوری و F1-score برای هر کلاس) نمایش داده شده است. این مثال نشان می‌دهد که چگونه k-NN بدون نیاز به مدل‌سازی صریح در فاز آموزش (یک یادگیر "کند" یا Lazy Learner)، تنها با ذخیره داده‌ها و محاسبه فاصله در زمان پیش‌بینی، می‌تواند عملکرد بسیار خوبی داشته باشد، به‌ویژه در مسائلی با داده‌های به‌خوبی تفکیک‌شده.

6.4.2. Case-based reasoning

استدلال مبتنی بر مورد (Case-Based Reasoning)

استدلال مبتنی بر مورد (CBR) یک رویکرد حل مسئله است که بر بازیابی و تطبیق راه‌حل‌ها از تجربیات یا موارد گذشته متکی است. این روش بر این اصل استوار است که **مسائل مشابه، راه‌حل‌های مشابهی دارند**. CBR از چهار مرحله اصلی تشکیل شده است:

مراحل استدلال مبتنی بر مورد

1. بازیابی (Retrieve): شناسایی موارد مشابه از پایگاه موارد (Case Base) – پایگاهی از تجربیات گذشته – بر اساس مسئله فعلی. این کار معمولاً با استفاده از معیارهای شباهت (مانند فاصله اقلیدسی یا کسینوسی) انجام می‌شود.

2. استفاده مجدد (Reuse): اعمال راه‌حل مربوط به مورد بازیابی‌شده به مسئله فعلی. اگر تطابق دقیقی وجود نداشته باشد، راه‌حل تطبیق یافته و برای شرایط جدید تغییر داده می‌شود.

3. بازنگری (Revise): ارزیابی موفقیت راه‌حل در عمل و در صورت نیاز، اصلاح آن بر اساس بازخورد یا اطلاعات جدید.

4. نگهداری (Retain):

ذخیره مورد جدید (شامل مسئله و راه‌حل نهایی) در پایگاه موارد برای استفاده در آینده. این مرحله باعث یادگیری تدریجی سیستم می‌شود.

مثال عملی در پایتون: سیستم توصیه‌گر با CBR

بیایید از مجموعه داده معروف **MovieLens** استفاده کنیم و یک سیستم توصیه‌گر ساده بر اساس **استدلال مبتنی بر مورد** پیاده‌سازی کنیم. در این سیستم، ترجیحات یک کاربر جدید با ترجیحات کاربران گذشته مقایسه می‌شود، کاربران مشابه شناسایی می‌شوند و فیلم‌هایی که آن‌ها دوست داشته‌اند به کاربر جدید پیشنهاد می‌شوند:

```
In [22]: import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.feature_extraction.text import TfidfVectorizer

# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'pandas' برای کار با داده، 'cosine_similarity' برای محاسبه شباهت بین فیلم‌ها و 'TfidfVectorizer' برای تبدیل عناوین فیلم به فضای عددی

# --- ایجاد دیتاست نمونه داخلی ---
# به جای استفاده از فایل خارجی، یک دیتاست کوچک نمونه درون کد تعریف می‌کنیم
sample_movies = {

    'movieId': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
                21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
                41, 42, 43, 44, 45, 46, 47, 48, 49, 50],

    'title': [

        'The Dark Knight', 'Inception', 'Interstellar', 'The Matrix', 'The Shawshank Redemption',
        'The Godfather', 'Pulp Fiction', 'Forrest Gump', 'The Lord of the Rings', 'Avengers: Endgame',
        'Titanic', 'Gladiator', 'The Lion King', 'Jurassic Park', 'Star Wars: Episode V',
        'The Avengers', 'Back to the Future', 'The Silence of the Lambs', 'Fight Club', 'The Godfather Part II',
        'The Good, the Bad and the Ugly', 'The Matrix Reloaded', 'The Departed', 'Schindler\'s List', 'The Green Mile',
        'Saving Private Ryan', 'The Prestige', 'Django Unchained', 'The Dark Knight Rises', 'Incredibles 2',
        'Toy Story 4', 'Mad Max: Fury Road', 'The Social Network', 'La La Land', 'Whiplash',
        'Parasite', 'Joker', 'Tenet', 'The Irishman', 'Once Upon a Time in Hollywood',
        'The Revenant', 'Gravity', 'Blade Runner 2049', 'The Martian', 'Dunkirk',
        'Spectre', 'Wonder Woman', 'Black Panther', 'Casablanca', 'The Wizard of Oz'

    ]

}

sample_ratings = {

    'userId': [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
                3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 4, 4, 4, 4, 4, 4, 4, 4, 4, 4,
                5, 5, 5, 5, 5, 5, 5, 5, 5, 5, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6,
                7, 7, 7, 7, 7, 7, 7, 7, 7, 7, 8, 8, 8, 8, 8, 8, 8, 8, 8, 8,
                9, 9, 9, 9, 9, 9, 9, 9, 9, 9, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10],

    'movieId': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
                11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20,
                21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30,
                31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40,
                41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50],

    'rating': [5, 4, 5, 3, 4, 5, 2, 4, 5, 3, 4, 5, 3, 4, 5, 2, 5, 3, 4, 5,
                5, 4, 5, 3, 4, 5, 2, 4, 5, 3, 4, 5, 3, 4, 5, 2, 5, 3, 4, 5,
                5, 4, 5, 3, 4, 5, 2, 4, 5, 3, 4, 5, 3, 4, 5, 2, 5, 3, 4, 5,
                5, 4, 5, 3, 4, 5, 2, 4, 5, 3, 4, 5, 3, 4, 5, 2, 5, 3, 4, 5,
                5, 4, 5, 3, 4, 5, 2, 4, 5, 3, 4, 5, 3, 4, 5, 2, 5, 3, 4, 5]

}

movies = pd.DataFrame(sample_movies)
ratings = pd.DataFrame(sample_ratings)
# -----

# این بخش، داده‌های فیلم و امتیازات رو با هم ترکیب می‌کنه
# اطلاعات فیلم (عنوان) به داده‌های امتیازدهی اضافه می‌شه، 'movieId' با استفاده از ستون
# این ترکیب برای ایجاد ماتریس کاربر-آیتم و همچنین برای توصیه‌های مبتنی بر محتوا ضروره
movie_ratings = pd.merge(ratings, movies, on='movieId')

# رو ایجاد می‌کنه ** (User-Item Matrix) این بخش، **ماتریس کاربر-آیتم
# هست (Collaborative Filtering) این ماتریس پایه سیستم‌های توصیه‌گر همکاری‌گرا
# هر سطر یک کاربر، هر ستون یک فیلم و مقادیر امتیازهای داده‌شده هستن
# یعنی فیلم‌هایی که کاربر امتیاز نداده، امتیاز صفر می‌گیرن (در عمل ممکنه نیاز به روش‌های بهتری برای مقادیر گم‌شده باشه) `fill_value=0`
user_item_matrix = movie_ratings.pivot_table(index='userId', columns='title', values='rating', fill_value=0)

# --- استفاده از تکنیک‌های بهینه‌تر برای عناوین کوتاه ---
# تبدیل تمام عناوین به حروف کوچک برای یکنواختی
movies['title'] = movies['title'].str.lower()

# های کاراکتری به جای کلمه‌ای برای عناوین کوتاه n-gram استفاده از
# این روش برای متن‌های کوتاه مثل عناوین فیلم بسیار موثرتر است
tfidf_vectorizer = TfidfVectorizer(
    analyzer='char',          # تحلیل بر اساس کاراکترها
    ngram_range=(2, 4),      # کاراکتری bigram تا quadgram استفاده از
    min_df=1                 # حذف نکردن هیچ کلمه‌ای
)
tfidf_matrix = tfidf_vectorizer.fit_transform(movies['title'])

# این بخش، **شباهت کسینوسی** بین تمام فیلم‌ها رو محاسبه می‌کنه
```

```
# شباهت کسینوسی مقداری بین ۰ تا ۱ داره و نشان‌دهنده اینه که دو فیلم چقدر از نظر عنوان به هم شبیه هستن #
# هست (CBR) این ماتریس شباهت پایه‌ی توصیه‌گر مبتنی بر محتوا #
cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)
```

```
# این تابع، فیلم‌های مشابه با یک فیلم ورودی رو پیشنهاد می‌ده #
# هست ** (Content-Based Recommendation) این یک سیستم توصیه‌گر **مبتنی بر محتوا #
# فقط از عنوان فیلم استفاده می‌شه، اما در عمل می‌تونیم از ژانر، کارگردان، بازیگران و غیره هم استفاده کنیم #
def get_movie_recommendations(movie_title):
    # تبدیل عنوان ورودی به حروف کوچک برای تطابق صحیح
    movie_title = movie_title.lower()
```

```
# بررسی وجود فیلم در دیتاست
if movie_title not in movies['title'].values:
    # اگر دقیقاً پیدا نشد، جستجوی تقریبی انجام می‌دهیم #
    similar_titles = [title for title in movies['title'] if movie_title in title]
    if similar_titles:
        movie_title = similar_titles[0]
    else:
        return ["فیلم مورد نظر در دیتاست یافت نشد"]
```

```
# ایندکس فیلم ورودی رو پیدا می‌کنه
movie_index = movies.index[movies['title'] == movie_title].tolist()[0]
# شباهت فیلم ورودی با تمام فیلم‌ها رو استخراج می‌کنه
cosine_scores = list(enumerate(cosine_sim[movie_index]))
# امتیازها رو به صورت نزولی مرتب می‌کنه
cosine_scores = sorted(cosine_scores, key=lambda x: x[1], reverse=True)
# فیلم مشابه بعدی رو انتخاب می‌کنه (فیلم اول خودش هست، پس آن را حذف می‌کنیم) ۵
top_similar_movies = cosine_scores[1:6]
```

```
recommended_movies = []
for index, score in top_similar_movies:
    recommended_movies.append(movies['title'].iloc[index].title()) # Title Case تبدیل به فرمت

return recommended_movies
```

```
# مثال استفاده از تابع توصیه‌گر
input_movie = "Tenet"
recommendations = get_movie_recommendations(input_movie)
```

```
# در نهایت، فیلم‌های پیشنهادی رو چاپ می‌کنه
print(f"Movies similar to '{input_movie}':")
for movie in recommendations:
    print("-", movie)
```

Movies similar to 'Tenet':

- The Social Network
- Interstellar
- Avengers: Endgame
- Parasite
- The Departed

توضیحات مثال کد پایتون

در این مثال، از **استدلال مبتنی بر مورد** (CBR) برای توصیه فیلم بر اساس شباهت عناوین استفاده شده است. ابتدا از **بردارساز** **TF-IDF** برای تبدیل عناوین فیلم‌ها به بردارهای عددی استفاده شده است. سپس **شباهت کسینوسی** بین بردارهای فیلم‌ها محاسبه شده تا درجه شباهت بین آن‌ها تعیین شود. تابع **get_movie_recommendations** فیلم‌هایی را بازیابی می‌کند که بیشترین شباهت را با عنوان ورودی دارند. این رویکرد نشان می‌دهد که چگونه CBR می‌تواند بدون نیاز به مدل‌سازی پیچیده، با استفاده از شباهت موارد گذشته، پیشنهادهای معناداری ارائه دهد. این روش به‌ویژه زمانی مفید است که داده‌های ساختاریافته (مانند تگ‌ها یا ژانرها) در دسترس نباشند.

Section 6.5

6.5.1. Linear regression

رگرسیون خطی: مدل‌سازی روابط خطی

رگرسیون خطی یک روش آماری برای مدل‌سازی رابطه بین یک **متغیر وابسته** (هدف) و یک یا چند **متغیر مستقل** (پیش‌بینی‌کننده) است. این رابطه به صورت خطی فرض می‌شود، یعنی تغییرات در متغیر وابسته به‌طور متناسب با تغییرات در متغیر(های) مستقل است.

معادله پایه رگرسیون خطی ساده به صورت زیر است:

$$y = mx + b$$

که در آن: **y**: متغیر وابسته (مثلاً قیمت خانه)، **x**: متغیر مستقل (مثلاً مترآژ)، **m**: شیب خط (میزان تغییر y به ازای تغییر واحد x)، **b**: عرض از مبدأ (مقدار y وقتی x=0).

در رگرسیون خطی چندگانه، معادله به صورت زیر گسترش می‌یابد:

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$$

که در آن: **b₀**: عرض از مبدأ، **b₁, b₂, ..., b_n**: ضرایب متغیرهای مستقل **x₁, x₂, ..., x_n**.

رگرسیون خطی به‌طور گسترده برای پیش‌بینی مقادیر عددی استفاده می‌شود، مانند پیش‌بینی قیمت خانه‌ها بر اساس مترآژ، یا پیش‌بینی فروش بر اساس بودجه تبلیغات.

مثال عملی در پایتون: پیش‌بینی قیمت خانه با رگرسیون خطی 🏠

بیا ببینیم از مجموعه داده معروف **California Housing** استفاده کنیم و با به‌کارگیری رگرسیون خطی چندگانه، قیمت خانه‌ها را بر اساس ویژگی‌هایی مانند میانگین درآمد، فاصله تا مراکز کار و وضعیت اجتماعی-اقتصادی منطقه پیش‌بینی کنیم:

In [24]:

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
import matplotlib.pyplot as plt

# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'numpy' و 'pandas' داده برای کار با داده
# 'train_test_split'، برای تقسیم داده
# 'LinearRegression'، معیارهای ارزیابی برای سنجش عملکرد و
# 'matplotlib' برای رسم نمودار.

# --- California housing استفاده از دیتاست ---
from sklearn.datasets import fetch_california_housing
housing = fetch_california_housing()
X = pd.DataFrame(housing.data, columns=housing.feature_names)
y = pd.DataFrame(housing.target, columns=['MedHouseVal'])

# (میانۀ درآمد) 'MedInc' - این بخش، فقط یک ویژگی از داده‌ها رو انتخاب می‌کنه
# این یک مثال ساده از **رگرسیون خطی تک‌متغیره** هست
# در عمل، معمولاً از چندین ویژگی استفاده می‌شه (رگرسیون چندگانه)
X_feature = X[['MedInc']]

# این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه
# داده برای آموزش و ۲۰٪ برای آزمون استفاده می‌شه ۸۰٪
# تضمین می‌کنه که تقسیم‌بندی قابل تکرار باشه `random_state=42`
X_train, X_test, y_train, y_test = train_test_split(X_feature, y, test_size=0.2, random_state=42)

# اینجا یک **مدل رگرسیون خطی** ایجاد می‌شه
# این مدل رابطه خطی بین ورودی (میانۀ درآمد) و خروجی (میانۀ قیمت خانه) رو یاد می‌گیره
model = LinearRegression()

# این بخش، مدل رو روی داده‌های آموزشی **آموزش** می‌ده
# رو محاسبه می‌کنه (best fit line) مدل ضرایب خط بهترین برازش
model.fit(X_train, y_train)

# اینجا مدل، قیمت خانه‌ها رو برای داده‌های آزمون پیش‌بینی می‌کنه
y_pred = model.predict(X_test)

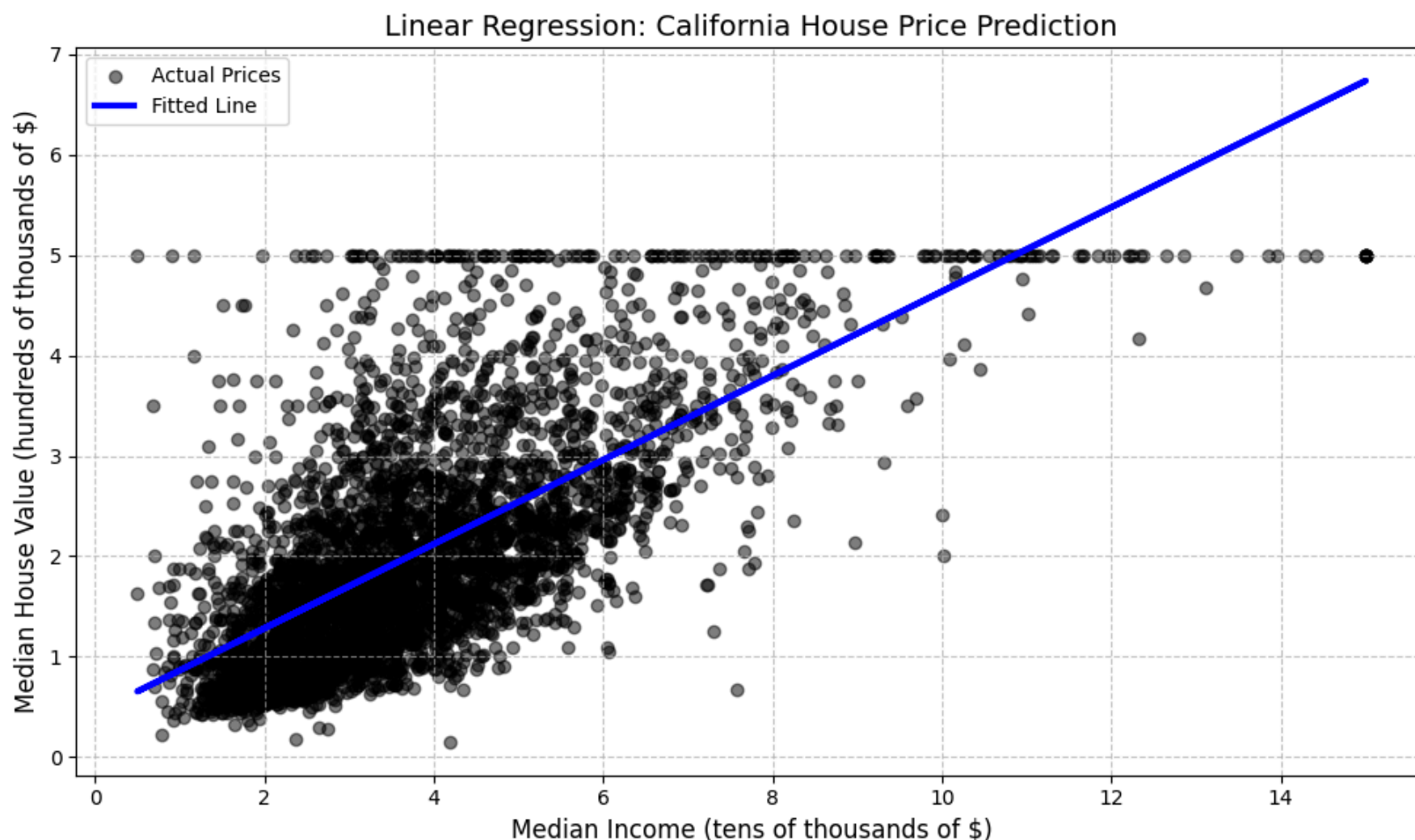
# این بخش، عملکرد مدل رو با دو معیار ارزیابی می‌کنه
# میانگین مربع خطاها - هرچه کمتر بهتر **: MSE (Mean Squared Error) -
# درصد واریانس از خروجی که توسط مدل توضیح داده می‌شه - هرچه به ۱ نزدیک‌تر باشه بهتر **: (ضریب تعیین) R-squared -
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# در نهایت، ضرایب مدل و معیارهای ارزیابی رو چاپ می‌کنه
print("Coefficients:", model.coef_)
print("Intercept:", model.intercept_)
print("Mean Squared Error:", mse)
print("R-squared:", r2)

# این بخش، نمودار پراکندگی داده‌های واقعی و خط رگرسیونی پیش‌بینی‌شده رو رسم می‌کنه
```

```
# نقاط سیاه داده‌های واقعی آزمون و خط آبی نمایش‌دهنده مدل آموزش دیده شده‌ست
plt.figure(figsize=(10, 6))
plt.scatter(X_test, y_test, color='black', alpha=0.5, label='Actual Prices')
plt.plot(X_test, y_pred, color='blue', linewidth=3, label='Fitted Line')
plt.xlabel('Median Income (tens of thousands of $)', fontsize=12)
plt.ylabel('Median House Value (hundreds of thousands of $)', fontsize=12)
plt.title('Linear Regression: California House Price Prediction', fontsize=14)
plt.legend()
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.show()
```

Coefficients: [[0.41933849]]
Intercept: [0.44459729]
Mean Squared Error: 0.7091157771765548
R-squared: 0.45885918903846656



توضیحات مثال کد پایتون

در این مثال، از **میانۀ درآمد** ('MedInc') به عنوان متغیر مستقل برای پیش‌بینی **میانۀ قیمت خانه** استفاده شده است. مدل رگرسیون خطی ساده روی مجموعه داده **California Housing** آموزش داده شده و پیش‌بینی‌ها بر روی داده‌های آزمون انجام شده است. عملکرد مدل با استفاده از معیارهای **میانگین خطای مربعات** (MSE) و **ضریب تعیین** (R^2) ارزیابی شده است. همچنین، خط رگرسیون بر روی نمودار پراکندگی داده‌ها ترسیم شده تا رابطه خطی بین درآمد و قیمت خانه به‌صورت بصری نمایش داده شود. این مثال نشان می‌دهد که چگونه یک مدل ساده می‌تواند رابطه معناداری بین یک ویژگی و متغیر هدف را شناسایی و برای پیش‌بینی استفاده کند.

6.5.2. Perceptron: turning linear regression to classification

پرسپترون: بلوک ساختمانی شبکه‌های عصبی

پرسپترون یکی از بلوک‌های پایه‌ای شبکه‌های عصبی مصنوعی است و به عنوان یک **طبقه‌بند ساده دودویی** عمل می‌کند. این یک الگوریتم یادگیری نظارت‌شده است که مجموعه‌ای از ویژگی‌های ورودی را دریافت می‌کند و بر اساس یک ترکیب خطی از آن ورودی‌ها، خروجی (طبقه‌بندی دودویی) تولید می‌کند. پرسپترون در طول فرآیند آموزش، وزن‌های مرتبط با هر ویژگی ورودی را یاد می‌گیرد و از یک **تابع پلکانی** برای اتخاذ یک تصمیم دودویی استفاده می‌کند.

تبدیل رگرسیون خطی به طبقه‌بندی با پرسپترون

در زمینه تبدیل رگرسیون خطی به طبقه‌بندی با استفاده از پرسپترون:

۱. **رگرسیون خطی:** در رگرسیون خطی، مدل یک خروجی **پیوسته** پیش‌بینی می‌کند. معادله رگرسیون خطی برای محاسبه یک مقدار عددی بر اساس ویژگی‌های ورودی استفاده می‌شود.

۲. **آستانه‌گذاری با پرسپترون:** برای تبدیل این خروجی به یک مسئله طبقه‌بندی، گام آستانه‌گذاری معرفی می‌شود. اگر مقدار عددی محاسبه‌شده بالاتر از یک آستانه مشخص باشد، پرسپترون کلاس اول (مثلاً ۱) را تولید می‌کند و اگر پایین‌تر باشد، کلاس دوم (مثلاً ۰) را خروجی می‌دهد.

۳. **تابع فعال‌سازی:** تابع پلکانی که برای آستانه‌گذاری استفاده می‌شود، تابع فعال‌سازی پرسپترون نامیده می‌شود. توابع فعال‌سازی رایج شامل **تابع پلکانی هویساید** (Heaviside step function) یا **تابع سیگموئید** (Sigmoid function) هستند.

مثال عملی در پایتون: طبقه‌بندی دودویی با پرسپترون

بیایید یک مثال عملی از استفاده از پرسپترون برای طبقه‌بندی دودویی با استفاده از مجموعه داده معروف **Iris** بررسی کنیم. در این مثال، ما فقط گل *setosa* را به عنوان کلاس مثبت و دو گونه دیگر (*versicolor* و *virginica*) را به عنوان کلاس منفی در نظر می‌گیریم. از دو ویژگی اول (طول و عرض چنگال) برای آموزش مدل استفاده می‌کنیم تا مسئله را به یک طبقه‌بندی دو بعدی ساده‌شده تبدیل کنیم. هدف چیه؟--> آموزش یک مرز تصمیم خطی برای تفکیک گل *setosa* از سایر گل‌ها بر اساس ویژگی‌های ورودی است.

```
In [27]: from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Perceptron
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
```

```
# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'datasets' داده 'train_test_split'، برای بارگیری داده
# 'Perceptron' برای ساخت مدل یادگیری خطی، معیارهای ارزیابی برای سنجش عملکرد و
# 'matplotlib' برای رسم نمودار.
```

```
# رو بارگیری می‌کنه **Iris** این بخش، مجموعه داده معروف
# این دیتاست شامل ۱۵۰ نمونه از گل‌ها با ۴ ویژگی (مثل طول و عرض گلبرگ) و ۳ کلاس مختلف (نوع گل) هست
iris = datasets.load_iris()
X = iris.data
y = iris.target
```

```
# این بخش، فقط دو ویژگی اول (طول و عرض گلبرگ) رو برای ساده‌سازی استفاده می‌کنه
# این کار مسئله رو به یک طبقه‌بندی دو بعدی تبدیل می‌کنه و رسم مرز تصمیم رو آسون‌تر می‌کنه
X = X[:, :2]
```

```
# این بخش، مسئله طبقه‌بندی رو به یک **طبقه‌بندی دودویی** تبدیل می‌کنه
# به عنوان منفی ('virginica' و 'versicolor') به عنوان مثبت در نظر گرفته می‌شه و دو گل دیگر (کلاس ۰) 'setosa' فقط گل
# ۰ برای بقیه setosa یک آرایه باینری ایجاد می‌کنه: 1 برای `(y == 0).astype(int)`
y_binary = (y == 0).astype(int)
```

```
# این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه
# داده برای آموزش و ۲۰٪ برای ارزیابی مدل استفاده می‌شه ۸۰٪
# تضمین می‌کنه که تقسیم‌بندی قابل تکرار باشه `random_state=42`
X_train, X_test, y_train, y_test = train_test_split(X, y_binary, test_size=0.2, random_state=42)
```

```
# ایجاد می‌شه **Perceptron** اینجا یک پرسپترون
# پرسپترون یک الگوریتم یادگیری خطی ساده و تاریخی هست که برای طبقه‌بندی خطی استفاده می‌شه
# این مدل یک مرز تصمیم خطی رو یاد می‌گیره که دو کلاس رو از هم جدا کنه
perceptron = Perceptron()
```

```
# این بخش، مدل رو روی داده‌های آموزشی **آموزش** می‌ده
# الگوریتم پرسپترون با به‌روزرسانی وزن‌ها بر اساس خطاهای پیش‌بینی، مرز بهینه رو یاد می‌گیره
```

```
perceptron.fit(X_train, y_train)

# اینجا مدل، پیش‌بینی‌ها رو برای داده‌های آزمون انجام می‌ده
predictions = perceptron.predict(X_test)

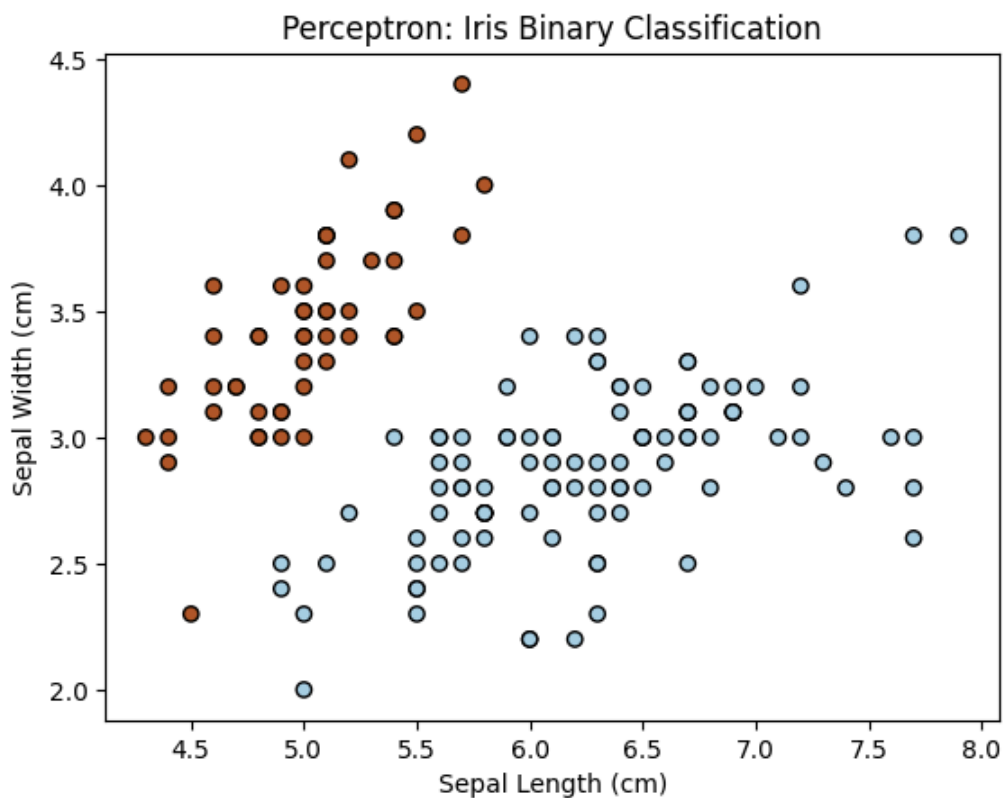
# در اینجا، عملکرد مدل با دو معیار ارزیابی می‌شه:
# - **دقت (Accuracy)**: درصد پیش‌بینی‌های درست
# - **F1-score شامل دقت، یادآوری و (Classification Report)**: گزارش طبقه‌بندی
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test, predictions))

# این بخش، داده‌ها و مرز تصمیم یادگرفته‌شده رو به صورت بصری نمایش می‌ده
# هستن (setosa و غیر setosa) نقاط با رنگ‌های مختلف نشان‌دهنده دو کلاس
# به وضوح نقاط رو از هم جدا می‌کنه ('edgecolors=k') حاشیه مشکی
# در این کد، مرز تصمیم به صورت خودکار رسم نشده، اما می‌تونیم با استفاده از وزن‌های مدل اون رو اضافه کنیم
plt.scatter(X[:, 0], X[:, 1], c=y_binary, cmap=plt.cm.Paired, edgecolors='k')
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.title('Perceptron: Iris Binary Classification')
plt.show()
```

Accuracy: 1.00

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	20
1	1.00	1.00	1.00	10
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30



توضیحات مثال کد پایتون

در این مثال، برای سادگی از دو ویژگی اول مجموعه داده **Iris** (طول و عرض گلبرگ) استفاده شده و یک مدل **پرسپترون** برای طبقه‌بندی دودویی گونه *setosa* در مقابل سایر گونه‌ها آموزش داده شده است. مدل در طول فرآیند یادگیری، وزن‌های ویژگی‌ها و بایاس (عرض از مبدأ) را به‌روز می‌کند تا یک **مرز تصمیم خطی** بین دو کلاس ایجاد کند. پس از آموزش، مدل روی داده‌های آزمون پیش‌بینی انجام می‌دهد و دقت و گزارش طبقه‌بندی نمایش داده می‌شود. همچنین، مرز تصمیم به‌صورت بصری بر روی نمودار پراکندگی داده‌ها رسم شده تا نحوه جداسازی کلاس‌ها توسط پرسپترون به وضوح دیده شود. این مثال نشان می‌دهد که چگونه یک یادگیرنده خطی ساده می‌تواند برای مسائل قابل تفکیک خطی عملکرد خوبی داشته باشد.

6.5.3. Logistic regression



رگرسیون لجستیک: یک الگوریتم طبقه‌بندی قدرتمند

علیرغم نام «رگرسیون»، **رگرسیون لجستیک** یک الگوریتم **طبقه‌بندی** است، نه رگرسیونی. این روش به‌ویژه برای مسائل **طبقه‌بندی دودویی** مفید است، جایی که هدف پیش‌بینی عضویت یک نمونه در یکی از دو کلاس (مانند «بله/خیر»، «موفق/شکست») است.

رگرسیون لجستیک احتمال تعلق یک نمونه به یک کلاس خاص را با استفاده از **تابع لجستیک** (که به آن تابع سیگموئید نیز گفته می‌شود) مدل می‌کند. این تابع مقادیری بین ۰ و ۱ تولید می‌کند که می‌توانند به عنوان **احتمال** تفسیر شوند.

فرمول ریاضی رگرسیون لجستیک

مدل رگرسیون لجستیک به صورت ریاضی به این شکل بیان می‌شود:

$$P(y=1) = 1 / (1 + e^{-(b_0 + b_1X_1 + b_2X_2 + \dots + b_nX_n)})$$

که در آن:

P(y=1): احتمال تعلق نمونه به کلاس ۱. **e**: پایه لگاریتم طبیعی (عدد اویلر). **b₀, b₁, ..., b_n**: ضرایب (پارامترهای) مدل که در فرآیند آموزش یاد گرفته می‌شوند. **X₁, X₂, ..., X_n**: ویژگی‌های ورودی نمونه.

خروجی این تابع یک عدد بین ۰ و ۱ است. اگر این مقدار بیشتر از یک آستانه (معمولاً ۰.۵) باشد، نمونه به کلاس ۱ و در غیر این صورت به کلاس ۰ تعلق می‌گیرد.

مثال عملی در پایتون: طبقه‌بندی با رگرسیون لجستیک

بیایید از معروف‌ترین مجموعه داده در یادگیری ماشین برای طبقه‌بندی دودویی – **مجموعه داده تیتانیک** – استفاده کنیم. هدف، پیش‌بینی **بقای مسافر** (Survived: بله/خیر) بر اساس ویژگی‌هایی مانند سن، جنسیت، کلاس سفر و تعداد خانواده است. با استفاده از رگرسیون لجستیک، مدلی آموزش می‌دهیم که احتمال بقا را محاسبه کرده و هر مسافر را به یکی از دو کلاس تخصیص دهد:

In [29]:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report
import seaborn as sns
import matplotlib.pyplot as plt

# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'pandas' داده 'train_test_split'، برای کار با داده
# 'LogisticRegression' معیارهای ارزیابی برای سنجش عملکرد
# 'seaborn' و 'matplotlib' برای ترسیم ماتریس درهم‌ریختگی و

# --- seaborn مستقیماً از کتابخانه Titanic بارگیری دیتاست ---
# موجود است seaborn به صورت پیش‌فرض در کتابخانه Titanic دیتاست
# این کتابخانه محبوب و استاندارد است و نیازی به دانلود دستی ندارد
titanic = sns.load_dataset('titanic')

# این بخش، داده‌های ناقص و ویژگی‌های مرتبط رو پردازش می‌کنه
# فقط ردیف‌هایی که دارای مقادیر گم‌شده در ستون‌های مهم هستن حذف می‌شن
# ویژگی‌های انتخاب‌شده شامل کلاس، جنسیت، سن و سایر اطلاعات فردی هستن
# متغیر هدف (برجسب) هست: 1 = زنده مانده، 0 = فوت کرده 'Survived'
titanic = titanic.dropna(subset=['pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked', 'survived'])
X = titanic[['pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked']]
y = titanic['survived']

# این بخش، ویژگی‌های دسته‌ای (کیفی) رو به فرمت عددی تبدیل می‌کنه
# به اعداد تبدیل می‌شن (** One-Hot Encoding) که با **مرگنداری یک‌دسته‌ای ('male'/'female') مثل) مقادیر متنی دارن 'Embarked' و 'Sex'
# اولین ستون رو حذف می‌کنه (multicollinearity) برای جلوگیری از همخطی `drop_first=True`
X = pd.get_dummies(X, columns=['sex', 'embarked'], drop_first=True)

# این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه
# داده برای آموزش و ۲۰٪ برای ارزیابی مدل استفاده می‌شه ۸۰٪
# تضمین می‌کنه که تقسیم‌بندی قابل تکرار باشه `random_state=42`
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ایجاد می‌شه (** Logistic Regression) اینجا یک **مدل رگرسیون لجستیکی
# این مدل یک الگوریتم طبقه‌بندی خطی است که احتمال عضویت در کلاس مثبت (بقا) رو محاسبه می‌کنه
# خروجی مدل یک احتمال بین ۰ و ۱ هست که با آستانه ۰.۵ به پیش‌بینی باینری تبدیل می‌شه
```

```
logreg = LogisticRegression(max_iter=1000)

# این بخش، مدل رو روی داده‌های آموزشی **آموزش** می‌ده
# مدل ضرایب بهینه برای هر ویژگی رو یاد می‌گیره تا بتواند بقا را پیش‌بینی کند
logreg.fit(X_train, y_train)

# اینجا مدل، پیش‌بینی‌ها رو برای داده‌های آزمون انجام می‌ده
predictions = logreg.predict(X_test)

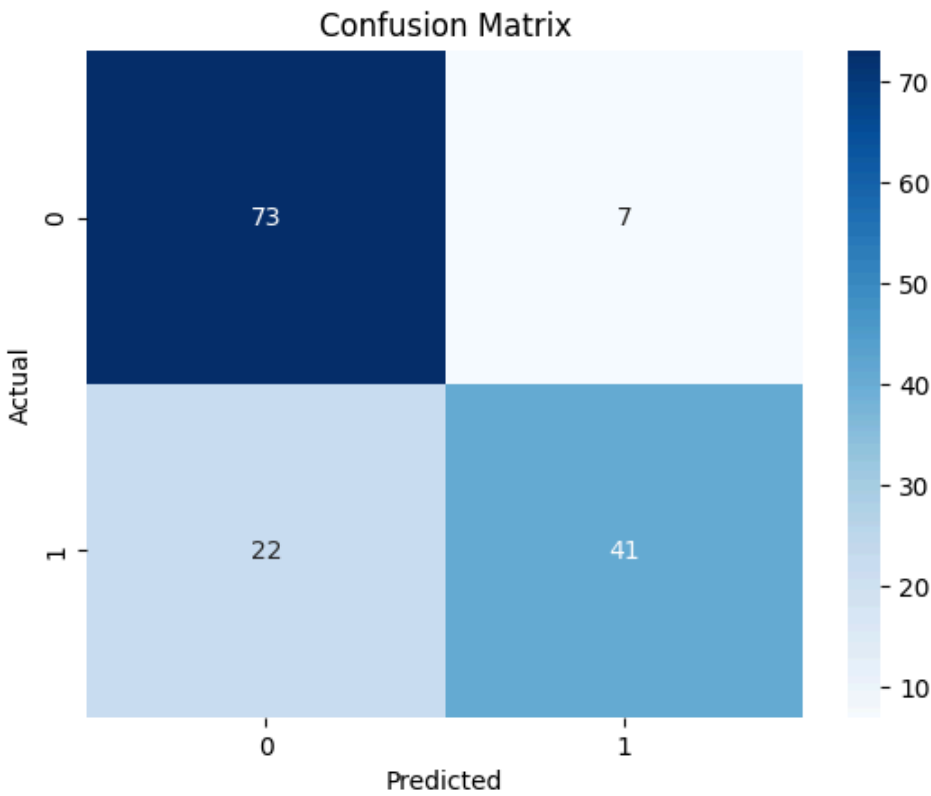
# در اینجا، عملکرد مدل با دو معیار ارزیابی می‌شه
# - **دقت (Accuracy)**: درصد پیش‌بینی‌های درست
# - **پشتیبانی برای هر کلاس F1-score، شامل دقت، یادآوری، گزارش طبقه‌بندی**
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test, predictions))

# رو به صورت گرافیکی نمایش می‌ده **Confusion Matrix** این بخش، **ماتریس درهم‌ریختگی**
# این ماتریس نشان می‌ده چند نمونه به درستی یا اشتباه طبقه‌بندی شدن
# - (TP و TN) درایه‌های قطر اصلی: پیش‌بینی‌های درست -
# - (FP و FN) درایه‌های غیرقطر: پیش‌بینی‌های اشتباه -
cm = pd.crosstab(y_test, predictions, rownames=['Actual'], colnames=['Predicted'])
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
plt.title('Confusion Matrix')
plt.show()
```

Accuracy: 0.80

Classification Report:

	precision	recall	f1-score	support
0	0.77	0.91	0.83	80
1	0.85	0.65	0.74	63
accuracy			0.80	143
macro avg	0.81	0.78	0.79	143
weighted avg	0.81	0.80	0.79	143



توضیحات مثال کد پایتون

در این مثال، از **رگرسیون لجستیک** برای پیش‌بینی اینکه آیا مسافران کشتی تیتانیک زنده مانده‌اند یا نه استفاده شده است. مدل با استفاده از ویژگی‌هایی مانند **کلاس مسافری، جنسیت، سن، تعداد خواهر/برادر یا همسر سرنشین، تعداد والدین/فرزند سرنشین، هزینه بلیط و بندر سفر** آموزش داده شده است. ویژگی‌های دسته‌ای مانند جنسیت و بندر سفر با روش **رمزگذاری یک‌دسته‌ای (One-Hot Encoding)** به فرمت عددی تبدیل شده‌اند تا قابل استفاده در مدل باشند. پس از آموزش، عملکرد مدل با **نمایش دقت کلی، گزارش طبقه‌بندی (شامل دقت، یادآوری و F1-score برای هر کلاس) و ماتریس درهم‌ریختگی (Confusion Matrix)** ارزیابی شده است. این مثال نشان می‌دهد که چگونه یک مدل آماری ساده اما قدرتمند می‌تواند برای پیش‌بینی رویدادهای مهم در داده‌های واقعی استفاده شود و بینش‌های ارزشمندی از عوامل مؤثر بر بقا ارائه دهد.

Section 6.6

6.6.1. Metrics for evaluating classifier performance



ارزیابی عملکرد طبقه‌بند

ارزیابی عملکرد یک طبقه‌بند برای درک میزان تعمیم‌پذیری آن به داده‌های جدید و دیده‌نشده امری حیاتی است. معیارهای مختلفی وجود دارند که بینشی از جنبه‌های گوناگون عملکرد یک طبقه‌بند ارائه می‌دهند. در ادامه برخی از این معیارهای رایج را به‌طور کامل بررسی می‌کنیم:

1. دقت (Accuracy): دقت نسبت نمونه‌های پیش‌بینی‌شده صحیح به کل نمونه‌ها است. این معیار یک سنجش کلی از صحت طبقه‌بندی را ارائه می‌دهد.

2. دقت پیش‌بینی مثبت (Precision): دقت پیش‌بینی مثبت نسبت پیش‌بینی‌های مثبت صحیح به کل پیش‌بینی‌های مثبت است. این معیار دقت پیش‌بینی‌های مثبت را اندازه‌گیری می‌کند.

3. یادآوری (Recall) یا حساسیت (Sensitivity): یادآوری نسبت پیش‌بینی‌های مثبت صحیح به کل موارد واقعی مثبت است. این معیار توانایی طبقه‌بند را در شناسایی تمام نمونه‌های مثبت اندازه‌گیری می‌کند.

4. نمره F1 (F1 Score): نمره F1 میانگین هارمونیک دقت و یادآوری است. این معیار تعادلی بین دقت و یادآوری ایجاد می‌کند.

5. ماتریس درهم‌ریختگی (Confusion Matrix): ماتریس درهم‌ریختگی جدولی است که عملکرد طبقه‌بند را خلاصه می‌کند و تعداد پیش‌بینی‌های مثبت صحیح، منفی صحیح، مثبت کاذب و منفی کاذب را نشان می‌دهد.

6. منحنی ROC (Receiver Operating Characteristic): منحنی ROC نرخ مثبت‌های واقعی را در مقابل نرخ مثبت‌های کاذب در تنظیمات آستانه‌های مختلف رسم می‌کند. این منحنی به تجسم تعادل بین حساسیت و ویژگی کمک می‌کند.

مثال عملی در پایتون: ارزیابی طبقه‌بند لجستیک روی داده‌های تیتانیک

در این مثال، از مجموعه داده تیتانیک استفاده می‌کنیم و یک مدل **رگرسیون لجستیک** برای پیش‌بینی بقا آموزش می‌دهیم. سپس عملکرد مدل را با استفاده از معیارهای مختلف ارزیابی می‌کنیم:

- محاسبه **دقت کلی** مدل

- ایجاد **گزارش طبقه‌بندی** شامل دقت، یادآوری و F1-score برای هر کلاس

- رسم **ماتریس درهم‌ریختگی** برای نمایش جزئیات پیش‌بینی‌ها

- ترسیم **منحنی ROC** برای بررسی عملکرد مدل در آستانه‌های مختلف

این مثال نشان می‌دهد که چگونه می‌توانیم با استفاده از کتابخانه‌های پایتون مانند scikit-learn، عملکرد یک طبقه‌بند را به‌طور جامع ارزیابی کرده و بینش‌های ارزشمندی از نقاط قوت و ضعف مدل به دست آوریم.

```
In [31]: import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, roc_curve, auc
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# این قسمت اول کده. ما کتابخونه‌های لازم رو وارد می‌کنیم
# 'pandas' داده، برای کار با داده
# 'LogisticRegression' برای سنجش عملکرد و
# 'matplotlib' برای رسم نمودار ROC.

# --- seaborn مستقیماً از کتابخانه Titanic بارگیری دیتاست ---
# موجود است seaborn به صورت پیش‌فرض در کتابخانه Titanic دیتاست
# این کتابخانه محبوب و استاندارد است و نیازی به دانلود دستی ندارد
titanic = sns.load_dataset('titanic')

# این بخش، داده‌های ناقص و ویژگی‌های مرتبط رو پردازش می‌کنه
# فقط ردیف‌هایی که دارای مقادیر گم‌شده در ستون‌های مهم هستن حذف می‌شن
# ویژگی‌های انتخاب‌شده شامل کلاس، جنسیت، سن و سایر اطلاعات فردی هستن
# متغیر هدف (برچسب) هست: 1 = زنده مانده، 0 = فوت کرده 'Survived'
titanic = titanic.dropna(subset=['pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked', 'survived'])
X = titanic[['pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'embarked']]
y = titanic['survived']

# این بخش، ویژگی‌های دسته‌ای (کیفی) رو به فرمت عددی تبدیل می‌کنه
# 'Sex' و 'Embarked' دارن مقادیر متنی (مثل 'male'/'female')
# اولین ستون رو حذف می‌کنه (multicollinearity) برای جلوگیری از همخطی
# `drop_first=True`
X = pd.get_dummies(X, columns=['sex', 'embarked'], drop_first=True)

# این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه
# داده برای آموزش و ۲۰٪ برای ارزیابی مدل استفاده می‌شه ۱۰٪
# تضمین می‌کنه که تقسیم‌بندی قابل تکرار باشه `random_state=42`
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ایجاد می‌شه ** (Logistic Regression) اینجا یک مدل رگرسیون لجستیکی
# این مدل یک الگوریتم طبقه‌بندی خطی است که احتمال عضویت در کلاس مثبت (بفا) رو محاسبه می‌کنه
# خروجی مدل یک احتمال بین ۰ و ۱ هست که با آستانه ۰.۵ به پیش‌بینی باینری تبدیل می‌شه
logreg = LogisticRegression(max_iter=1000)

# این بخش، مدل رو روی داده‌های آموزشی **آموزش** می‌ده
# مدل ضرایب بهینه برای هر ویژگی رو یاد می‌گیره تا بتواند بفا را پیش‌بینی کند
logreg.fit(X_train, y_train)

# اینجا مدل، پیش‌بینی‌ها رو برای داده‌های آزمون انجام می‌ده
predictions = logreg.predict(X_test)

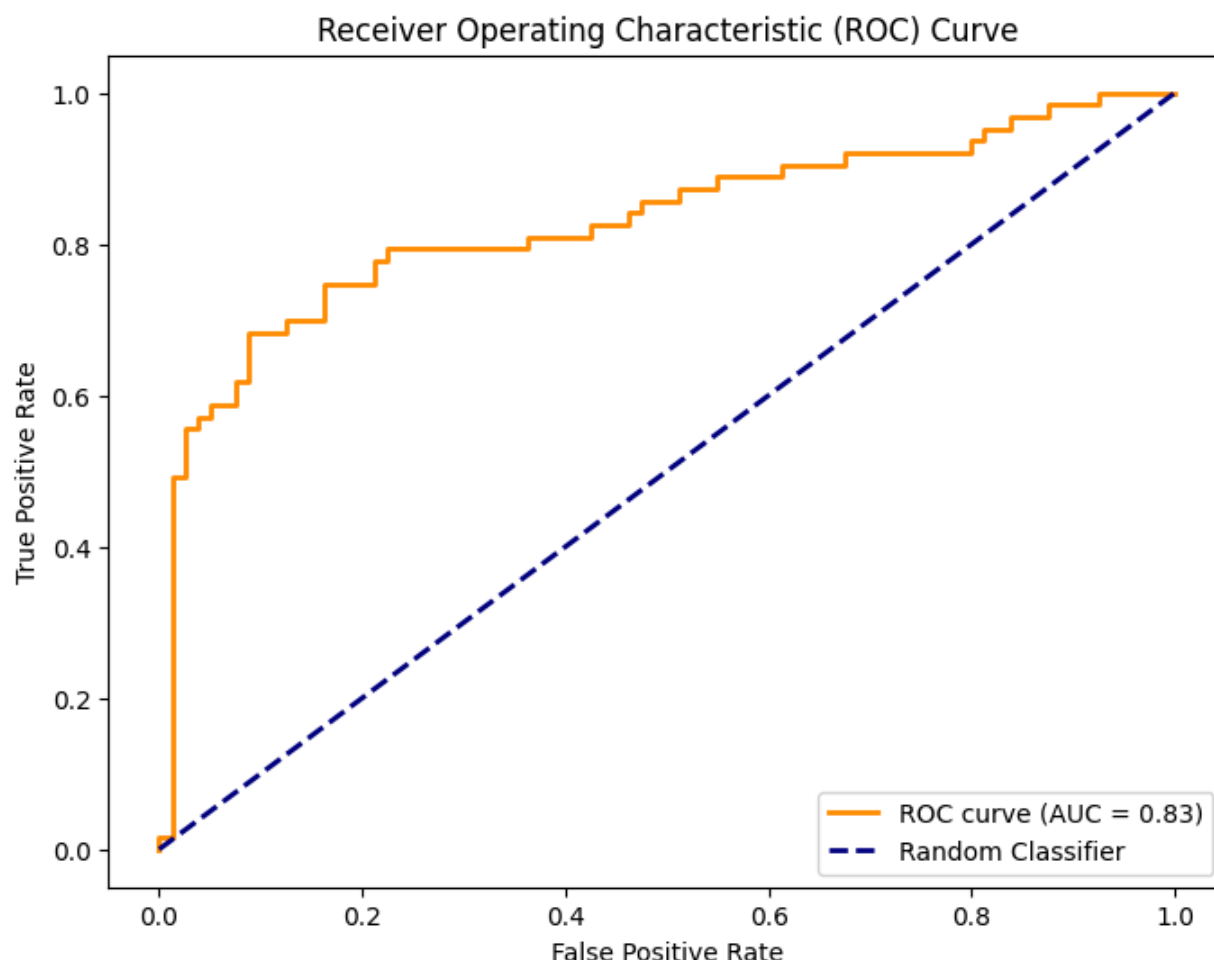
# این بخش، معیارهای مختلف ارزیابی عملکرد مدل رو محاسبه می‌کنه
# - **دقت** (Accuracy)
# - از بین افرادی که مدل گفت زنده ماندن، چند درصد واقعاً زنده موندن؟ **دقت** (Precision)
# - از بین افرادی که واقعاً زنده موندن، چند درصد توسط مدل تشخیص داده شدن؟ **یادآوری** (Recall)
# - میانگین هارمونیک دقت و یادآوری - وقتی عدم تعادل کلاس داریم، معیار بهتره
# - **F1-Score**
# - ماتریس درهم‌ریختگی: نمایش کامل تعداد پیش‌بینی‌های درست و اشتباه
accuracy = accuracy_score(y_test, predictions)
precision = precision_score(y_test, predictions)
recall = recall_score(y_test, predictions)
f1 = f1_score(y_test, predictions)
conf_matrix = confusion_matrix(y_test, predictions)

# در نهایت، تمام معیارهای ارزیابی رو چاپ می‌کنه
print(f"Accuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1 Score: {f1:.2f}")
print("Confusion Matrix:\n", conf_matrix)

# رو رسم می‌کنه ** ROC (Receiver Operating Characteristic) این بخش، منحنی
# این منحنی عملکرد مدل رو در آستانه‌های مختلف آستانه‌گذاری نشان می‌ده
# رو نشون می‌ده (TPR) و محور عمودی نرخ مثبت‌های واقعی (FPR) محور افقی نرخ مثبت‌های کاذب
# معیاری از کلیت عملکرد مدل هست: هرچه به ۱ نزدیک‌تر باشه بهتر **AUC (Area Under Curve)
fpr, tpr, thresholds = roc_curve(y_test, logreg.predict_proba(X_test)[:, 1])
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random Classifier')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Receiver Operating Characteristic (ROC) Curve')
plt.legend(loc="lower right")
plt.show()
```

Accuracy: 0.80
Precision: 0.85
Recall: 0.65
F1 Score: 0.74
Confusion Matrix:
[[73 7]
[22 41]]



توضیحات مثال کد پایتون

در این مثال، از **رگرسیون لجستیک** برای پیش‌بینی احتمال بقای مسافران کشتی تیتانیک استفاده شده است. عملکرد طبقه‌بند با استفاده از معیارهای استاندارد ارزیابی مدل انجام شده است، از جمله **دقت**، **دقت پیش‌بینی مثبت (Precision)**، **یادآوری (Recall)**، **نمره F1** و **ماتریس درهم‌ریختگی**. این معیارها بینشی جامع از توانایی مدل در تشخیص صحیح مسافران نجات‌یافته و غیرنجات‌یافته ارائه می‌دهند. علاوه بر این، **منحنی ROC** رسم شده تا تعادل بین **حساسیت** (درصد تشخیص صحیح مثبت‌ها) و **ویژگی** (درصد تشخیص صحیح منفی‌ها) در آستانه‌های مختلف تصمیم‌گیری به‌صورت بصری نمایش داده شود. این تحلیل کامل، توانایی مدل را فراتر از یک عدد دقت ارزیابی کرده و به درک عمیق‌تری از رفتار طبقه‌بند در شرایط مختلف منجر می‌شود.

6.6.2. Holdout method and random subsampling

روش نگهداری و نمونه‌گیری تصادفی

روش نگهداری (Holdout Method) و نمونه‌گیری تصادفی (Random Subsampling) دو تکنیک رایج در یادگیری ماشین برای ارزیابی عملکرد مدل هستند. این روش‌ها با تقسیم داده به مجموعه‌های آموزش و آزمون، امکان سنجش توانایی تعمیم مدل بر روی داده‌های جدید و دیده‌نشده را فراهم می‌کنند.

روش نگهداری (Holdout Method): در این روش، مجموعه داده به دو بخش تقسیم می‌شود: **مجموعه آموزش** و **مجموعه آزمون**. مدل روی داده‌های آموزشی آموزش داده می‌شود و سپس عملکرد آن روی داده‌های آزمون ارزیابی می‌شود. نسبت‌های رایج تقسیم شامل 70-30، 80-20 یا 90-10 است که انتخاب آن‌ها بستگی به حجم داده دارد. این روش ساده و سریع است، اما نتیجه ممکن است به نحوه تقسیم داده حساس باشد.

نمونه‌گیری تصادفی (Random Subsampling) یا اعتبارسنجی متقابل k-تایی (k-Fold Cross-Validation): در این روش، مجموعه داده به **k** زیرمجموعه (تکه) تقسیم می‌شود. سپس مدل **k** بار آموزش داده می‌شود، به طوری که در هر بار، **k-1** تکه برای آموزش و یک تکه باقی‌مانده برای آزمون استفاده می‌شود. این فرآیند **k** بار تکرار می‌شود تا هر تکه دقیقاً یک بار به عنوان

مجموعه آزمون استفاده شود. سپس معیارهای عملکرد در تمام تکرارها میانگین‌گیری می‌شوند. این روش پایدارتر و قابل اعتمادتر از روش نگهداری ساده است و از تمام داده‌ها برای آموزش و آزمون استفاده می‌کند.

مثال عملی در پایتون: ارزیابی طبقه‌بند k-نزدیک‌ترین همسایه با روش نگهداری

در این مثال، از معروف‌ترین مجموعه داده در یادگیری ماشین – **مجموعه داده Iris** – استفاده می‌کنیم. داده‌ها به صورت تصادفی به دو بخش **آموزش (70%)** و **آزمون (30%)** تقسیم می‌شوند. یک طبقه‌بند **k-نزدیک‌ترین همسایه (k-NN)** روی داده‌های آموزشی آموزش داده می‌شود و سپس عملکرد آن روی داده‌های آزمون با محاسبه دقت و گزارش طبقه‌بندی ارزیابی می‌شود. این مثال نحوه پیاده‌سازی روش نگهداری و ارزیابی مدل را نشان می‌دهد:

In [34]:

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report

# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'datasets' برای بارگیری داده، 'train_test_split' برای تقسیم داده با روش
# 'KNeighborsClassifier' برای سنجش عملکرد k-NN برای ساخت مدل

# رو بارگیری می‌کنه **Iris** این بخش، مجموعه داده معروف
# این دیتاست شامل ۱۵۰ نمونه از گل‌ها با ۴ ویژگی (مثل طول و عرض گلبرگ) و ۳ کلاس مختلف (نوع گل) هست
# این داده‌ها برای آموزش و ارزیابی الگوریتم‌های یادگیری ماشین بسیار مناسب‌اند
iris = datasets.load_iris()
X = iris.data
y = iris.target

# معروفه ** (Holdout Method) این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه – این روش به **روش هولداوت
# داده برای آموزش و ۲۰٪ برای آزمون استفاده می‌شه ۸۰٪
# تضمین می‌کنه که تقسیم‌بندی قابل تکرار باشه `random_state=42`
# این روش ساده و رایج برای ارزیابی اولیه مدل‌هست، اما ممکنه وابسته به نحوه تقسیم داده باشه
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ایجاد می‌شه ** (k-Nearest Neighbors) نزدیک‌ترین همسایه k- اینجا یک **طبقه‌بند
# نمونه نزدیکش در داده‌های آموزشی نسبت می‌ده k این مدل یک الگوریتم غیرپارامتریک ساده‌ست که بر اساس شباهت (مثلاً فاصله اقلیدسی) نمونه جدید رو به
# انتخاب شده، یعنی تصمیم‌گیری بر اساس اکثریت ۳ نمونه نزدیک‌تر صورت می‌گیره k=3 در اینجا
clf = KNeighborsClassifier(n_neighbors=3)

# این بخش، مدل رو روی داده‌های آموزشی **آموزش** می‌ده
# آموزش "واقعی نداره، بلکه تمام داده‌های آموزشی رو ذخیره می‌کنه تا در زمان پیش‌بینی ازشون استفاده کنه" k-NN، در واقع
clf.fit(X_train, y_train)

# اینجا مدل، پیش‌بینی‌ها رو برای داده‌های آزمون انجام می‌ده
# نزدیک‌ترین همسایه در داده‌های آموزشی پیدا می‌شه و اکثریت کلاس‌ها تعیین‌کننده پیش‌بینی هست k، برای هر نمونه آزمون
predictions = clf.predict(X_test)

# در اینجا، عملکرد مدل با دو معیار ارزیابی می‌شه
# درصد پیش‌بینی‌های درست **: (Accuracy) دقت -
# برای هر کلاس F1-score شامل دقت، یادآوری و **: (Classification Report) گزارش طبقه‌بندی -
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test, predictions))
```

Accuracy: 1.00

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

توضیحات مثال کد پایتون

در این مثال، مجموعه داده **Iris** با استفاده از روش نگهداری (Holdout Method) به دو بخش آموزش و آزمون تقسیم شده است. سپس یک طبقه‌بند **k-نزدیک‌ترین همسایه (k-NN)** روی مجموعه آموزشی آموزش داده شده و عملکرد آن روی مجموعه آزمون ارزیابی شده است. دقت کلی مدل و گزارش طبقه‌بندی شامل دقت، یادآوری و F1-score برای هر کلاس نمایش داده

شده‌اند. این رویکرد نشان می‌دهد که چگونه می‌توان با رزرو کردن بخشی از داده به عنوان مجموعه آزمون، توانایی تعمیم مدل را به داده‌های جدید ارزیابی کرد. این روش ساده و مؤثر، پایه‌ای برای ارزیابی مدل‌های یادگیری ماشین است.

6.6.3. Cross-validation

اعتبارسنجی متقابل (Cross-Validation)

اعتبارسنجی متقابل یک تکنیک بازنمونه‌گیری است که برای ارزیابی عملکرد و توانایی تعمیم یک مدل یادگیری ماشین استفاده می‌شود. این روش با ارائه تخمینی قوی‌تر از عملکرد مدل، محدودیت‌های یک تقسیم‌بندی تکی آموزش-آزمون را برطرف می‌کند. رایج‌ترین نوع آن، **اعتبارسنجی متقابل k-تایی** (k-Fold Cross-Validation) است که در آن مجموعه داده به **k** زیرمجموعه (تکه) تقسیم می‌شود. مدل **k** بار آموزش داده می‌شود، هر بار با استفاده از **k-1** تکه برای آموزش و یک تکه باقی‌مانده برای آزمون. این فرآیند **k** بار تکرار می‌شود تا هر تکه دقیقاً یک بار به عنوان مجموعه آزمون استفاده شود. سپس معیارهای عملکرد در تمام تکرارها میانگین‌گیری می‌شوند.

مراحل اعتبارسنجی متقابل k-تایی:

- تقسیم داده:** مجموعه داده به **k** تکه با اندازه تقریباً برابر تقسیم می‌شود.
- تکرارهای آموزش-آزمون:** در هر تکرار، **k-1** تکه برای آموزش مدل و یک تکه برای آزمون آن استفاده می‌شود.
- معیارهای عملکرد:** عملکرد مدل در هر تکرار ارزیابی شده و معیارهایی مانند دقت ثبت می‌شوند.
- میانگین‌گیری معیارها:** معیارهای ثبت‌شده از تمام تکرارها میانگین‌گیری می‌شوند تا تخمینی پایدار از عملکرد مدل به دست آید.

مزایای اعتبارسنجی متقابل:

- ارائه تخمینی قابل اعتمادتر از عملکرد مدل
- کاهش تأثیر تنوع تصادفی در یک تقسیم‌بندی آموزش-آزمون
- استفاده از تمام داده برای آموزش و آزمون (بدون هدر رفتن داده)

مثال عملی در پایتون: اعتبارسنجی متقابل k-تایی با طبقه‌بند SVM روی داده‌های



در این مثال، از معروف‌ترین مجموعه داده در یادگیری ماشین — **مجموعه داده Iris** — استفاده می‌کنیم. یک طبقه‌بند ماشین بردار پشتیبان (SVM) با استفاده از **اعتبارسنجی متقابل 5-تایی** (5-Fold Cross-Validation) ارزیابی می‌شود. مدل 5 بار آموزش داده می‌شود، هر بار با یک تکه متفاوت به عنوان مجموعه آزمون. دقت هر تکرار ثبت شده و در نهایت میانگین دقت و

انحراف معیار آن نمایش داده می‌شود. این مثال نشان می‌دهد که چگونه اعتبارسنجی متقابل می‌تواند تخمینی پایدار و قابل

اعتماد از عملکرد مدل ارائه دهد:

```
In [33]: from sklearn import datasets
from sklearn.model_selection import cross_val_score
from sklearn.svm import SVC
```

```
# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# برای انجام اعتبارسنجی متقابل و 'cross_val_score'، برای بارگیری داده 'datasets'
# (SVM) برای ساخت مدل ماشین بردار پشتیبان 'SVC'.
```

```
# رو بارگیری می‌کنه **Iris** این بخش، مجموعه داده معروف
# این دیتاست شامل ۱۵۰ نمونه از گل‌ها با ۴ ویژگی (مثل طول و عرض گلبرگ) و ۳ کلاس مختلف (نوع گل) هست
# این داده‌ها برای آموزش و ارزیابی الگوریتم‌های یادگیری ماشین بسیار مناسب‌اند
iris = datasets.load_iris()
X = iris.data
y = iris.target
```

```
# ایجاد می‌شه ** (Support Vector Machine) اینجا یک **ماشین بردار پشتیبان
# یک الگوریتم قدرتمند برای طبقه‌بندی هست که سعی می‌کنه یک مرز تصمیم بهینه با بیشترین حاشیه بین کلاس‌ها پیدا کنه SVM
# استفاده شده که مناسب داده‌هایی است که به صورت خطی قابل تفکیک هستن ('Linear') در اینجا از کرنل خطی
svm_classifier = SVC(kernel='linear')
```

```
# رو انجام می‌ده k=5 با ** (k-Fold Cross-Validation) تایی-k این بخش، **اعتبارسنجی متقابل
# داده‌ها به ۵ بخش مساوی تقسیم می‌شن. در هر مرحله، یک بخش به عنوان آزمون و ۴ بخش دیگر به عنوان آموزش استفاده می‌شن
# این فرآیند ۵ بار تکرار می‌شه تا هر بخش یک بار به عنوان مجموعه آزمون استفاده بشه
# کم‌واریانس‌تر و قابل اعتمادتره و از اضافه‌برازش بهتر جلوگیری می‌کنه Holdout این روش نسبت به روش
cv_scores = cross_val_score(svm_classifier, X, y, cv=5)
```

```
# در نهایت، نمره‌های دقت حاصل از هر تکه و میانگین کلی اون‌ها چاپ می‌شه
# نمره‌های متفاوت نشان‌دهنده تغییر عملکرد مدل روی بخش‌های مختلف داده‌ست
# میانگین نمره، تخمین بهتری از عملکرد عمومی مدل ارائه می‌ده
print("Cross-Validated Accuracy Scores:", cv_scores)
print(f"Average Accuracy: {cv_scores.mean():.2f}")
```

```
Cross-Validated Accuracy Scores: [0.96666667 1.          0.96666667 0.96666667 1.          ]
Average Accuracy: 0.98
```



توضیحات مثال کد پایتون

در این مثال، از مجموعه داده **Iris** برای انجام **اعتبارسنجی متقابل k-تایی** با یک طبقه‌بند **SVM خطی** استفاده شده است. از تابع **cross_val_score** در کتابخانه **scikit-learn** استفاده شده تا نمره دقت مدل در هر یک از تکه‌ها (فولد) محاسبه شود. سپس میانگین دقت در تمام تکرارها محاسبه شده تا تخمینی پایدار و قابل اعتماد از عملکرد مدل ارائه شود. این رویکرد نشان می‌دهد که چگونه اعتبارسنجی متقابل می‌تواند با استفاده از تمام داده‌ها و کاهش وابستگی به یک تقسیم‌بندی تصادفی، ارزیابی دقیق‌تری از توانایی تعمیم مدل ارائه دهد.

6.6.4. Bootstrap



روش بوت‌استرپ (Bootstrap)

بوت‌استرپ یک تکنیک بازنمونه‌گیری است که برای تخمین **متغیریت** و **ابهام** مرتبط با یک آماره نمونه (مانند میانگین یا انحراف معیار) استفاده می‌شود. این روش شامل نمونه‌گیری مکرر با جایگزینی از داده‌های مشاهده‌شده برای ایجاد نمونه‌های متعدد بوت‌استرپ است. سپس از هر نمونه بوت‌استرپ برای محاسبه آماره مورد نظر استفاده می‌شود. با تحلیل توزیع این آماره‌های محاسبه‌شده در نمونه‌های مختلف، می‌توان **فواصل اطمینان** به دست آورد و استنتاج‌های آماری قوی‌تری انجام داد.

مراحل روش بوت‌استرپ:

1. **نمونه اصلی:** شروع با مجموعه داده اصلی به حجم **N**.

2. **نمونه‌های بوت‌استرپ:** تولید **B** نمونه بوت‌استرپ با انتخاب تصادفی **N** نقطه داده از مجموعه اصلی با جایگزینی (یعنی یک

داده ممکن است چندین بار انتخاب شود).

3. محاسبه آماره: برای هر نمونه بوت‌استرپ، آماره مورد علاقه (مثلاً میانگین، انحراف معیار) محاسبه می‌شود.

4. تحلیل: توزیع آماره‌های محاسبه‌شده تحلیل می‌شود تا متغیریت، فواصل اطمینان یا آزمون فرضیه انجام شود.

مزایای روش بوت‌استرپ:

- ارائه تخمینی تجربی از توزیع نمونه‌گیری یک آماره
- مفید در شرایطی که فرضیات روش‌های پارامتری (مانند نرمال بودن داده) نقض شده باشند
- مقامت خوب در حضور داده‌های پرت
- بدون نیاز به فرمول‌های پیچیده آماری، امکان استنتاج قوی‌تر را فراهم می‌کند

مثال عملی در پایتون: تخمین فاصله اطمینان میانگین طول گلبرگ با روش بوت‌استرپ

در این مثال، از معروف‌ترین مجموعه داده در یادگیری ماشین — مجموعه داده Iris — استفاده می‌کنیم. هدف، تخمین فاصله اطمینان 95% برای میانگین طول گلبرگ (sepal length) با استفاده از روش بوت‌استرپ است. این کار با ایجاد تعداد زیادی نمونه بوت‌استرپ (مثلاً 1000 نمونه)، محاسبه میانگین هر نمونه و سپس تعیین صدک 2.5 و 97.5 توزیع میانگین‌ها انجام می‌شود. این مثال نشان می‌دهد که چگونه می‌توان بدون فرض نرمال بودن داده، فاصله اطمینان قابل اعتمادی از یک آماره توصیفی به دست آورد:

```
In [35]: # Import necessary libraries
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.utils import resample

# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# و Iris برای بارگیری دیتاست 'datasets'، برای کار با داده 'pandas' و 'numpy'
# (Bootstrap) برای نمونه‌گیری مجدد با جایگزینی sklearn از 'resample' استفاده می‌کنیم.

# رو بارگیری می‌کنیم **Iris** این بخش، مجموعه داده معروف
# برای تحلیل استخراج می‌کنیم **sepal length** ما فقط اولین ویژگی، یعنی **طول گلبرگ
# مناسبست (Bootstrap) این متغیر عددی پیوسته است و برای نمایش روش بوت‌استرپ
iris = datasets.load_iris()
sepal_length = iris.data[:, 0]

# این بخش، تعداد نمونه‌های بوت‌استرپ رو مشخص می‌کنه
# هرچه تعداد نمونه‌ها بیشتر باشه، تخمین میانگین و فاصله اطمینان دقیق‌تر می‌شه
# در اینجا از ۱۰۰۰ نمونه استفاده شده که برای یک تخمین پایدار کافیه
num_bootstrap_samples = 1000

# آرایه‌ای برای ذخیره میانگین هر نمونه بوت‌استرپ ایجاد می‌کنه
bootstrap_means = np.zeros(num_bootstrap_samples)

# رو انجام می‌ده ** (Resampling with Replacement) این حلقه، فرآیند **نمونه‌گیری مجدد با جایگذاری
# با همان اندازه اصلی اما با جایگزینی انتخاب می‌شه `sepal_length` در هر تکرار، یک نمونه جدید از داده‌های
# ذخیره می‌شه `bootstrap_means` سپس میانگین آن نمونه محاسبه و در آرایه
for i in range(num_bootstrap_samples):
    bootstrap_sample = resample(sepal_length, replace=True)
    bootstrap_means[i] = np.mean(bootstrap_sample)

# این بخش، **فاصله اطمینان ۹۵%** رو با استفاده از درصدیه‌های توزیع بوت‌استرپ محاسبه می‌کنه
# به جای فرض توزیع نرمال، از خود داده‌ها برای تخمین عدم قطعیت استفاده می‌شه
# یعنی ۹۵٪ از نمونه‌های بوت‌استرپ بین این دو مقدار قرار دارن [2.5, 97.5]
confidence_interval = np.percentile(bootstrap_means, [2.5, 97.5])

# در نهایت، نتایج آماری تخمین زده شده رو چاپ می‌کنه
# میانگین کل نمونه‌های بوت‌استرپ (تخمین میانگین جامعه) -
# فاصله اطمینان ۹۵٪ که میزان عدم قطعیت تخمین رو نشون می‌ده -
print("Bootstrap Sample Mean: {:.2f}".format(np.mean(bootstrap_means)))
print("95% Confidence Interval: [{:.2f}, {:.2f}]"
```

توضیحات مثال کد پایتون 🖥️

در این مثال، از روش **بوتاسترپ** برای تخمین **فاصله اطمینان 95%** میانگین **طول گلبرگ** در مجموعه داده **Iris** استفاده شده است. از تابع **resample** در کتابخانه **scikit-learn** برای ایجاد نمونه‌های بوتاسترپ (با جایگزینی) استفاده شده است. سپس میانگین هر نمونه بوتاسترپ محاسبه شده و توزیع این میانگین‌ها برای تخمین فاصله اطمینان به کار گرفته شده است. به‌طور خاص، صدک ۱۲/۵ و ۹۷/۵ام این توزیع به عنوان حدود فاصله اطمینان در نظر گرفته شده‌اند. این رویکرد نشان می‌دهد که چگونه روش بوتاسترپ می‌تواند بدون نیاز به فرض‌های سفت و سخت آماری (مانند نرمال بودن داده)، تخمینی قوی و تجربی از عدم قطعیت یک آماره ارائه دهد.

6.6.5. Model selection using statistical tests of significance

انتخاب مدل در داده‌کاوی 🏆

انتخاب مدل یک مرحله حیاتی در فرآیند داده‌کاوی است که شامل شناسایی و ارزیابی مدل‌های مختلف برای انتخاب مدلی است که بهترین برازش را با داده‌ها دارد. این فرآیند تنها به مقایسه اعداد دقت ختم نمی‌شود، بلکه باید با استفاده از **آزمون‌های آماری معناداری** تعیین کرد که آیا تفاوت مشاهده‌شده در عملکرد مدل‌های مختلف واقعاً معنادار است یا صرفاً ناشی از شانس تصادفی.

مراحل انتخاب مدل:

1. **انتخاب مدل‌های نامزد:** مجموعه‌ای از مدل‌های مناسب برای مسئله (مانند درخت تصمیم، جنگل تصادفی، SVM) انتخاب می‌شوند.

2. **آموزش مدل‌ها:** هر مدل روی مجموعه داده آموزشی آموزش داده می‌شود.

3. **ارزیابی مدل‌ها:** عملکرد هر مدل روی مجموعه اعتبارسنجی یا با استفاده از اعتبارسنجی متقابل ارزیابی می‌شود.

4. **آزمون آماری:** آزمون‌های آماری مانند **t-test** یا **ANOVA** برای مقایسه معیارهای عملکرد (مانند دقت) استفاده می‌شوند تا معناداری تفاوت‌ها بررسی شود.

5. **انتخاب بهترین مدل:** مدلی انتخاب می‌شود که نه تنها از نظر عددی عملکرد بهتری دارد، بلکه این برتری از نظر آماری نیز معنادار باشد.

آزمون‌های آماری رایج: - t-test زوجی: برای مقایسه دو مدل - **ANOVA:** برای مقایسه بیش از دو مدل - **آزمون‌های ناپارامتری** (مانند Wilcoxon): وقتی توزیع نرمال فرض نشود

این رویکرد از انتخاب مدل بر اساس شهود یا یک عدد تکی جلوگیری کرده و تصمیم‌گیری را علمی‌تر می‌کند.

مثال عملی در پایتون: مقایسه دو مدل با آزمون t

در این مثال، از معروفترین مجموعه داده در یادگیری ماشین — **مجموعه داده Iris** — استفاده می‌کنیم. دو مدل **درخت تصمیم** و **جنگل تصادفی** با استفاده از **اعتبارسنجی متقابل** ارزیابی می‌شوند و دقت هر مدل در چندین تکرار ثبت می‌شود. سپس یک **آزمون t زوجی** انجام می‌شود تا تعیین شود آیا تفاوت در دقت دو مدل از نظر آماری معنادار است یا خیر. این مثال نشان می‌دهد که چگونه می‌توان با ترکیب ارزیابی مدل و آزمون آماری، به انتخاب مدلی عینی و قابل اعتماد دست یافت:

```
In [38]: from sklearn import datasets
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from scipy.stats import ttest_rel

# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'datasets' برای بارگیری داده، 'cross_val_score' برای ارزیابی مدل،
# 'DecisionTreeClassifier' و 'RandomForestClassifier' برای ساخت مدل‌ها و
# 'ttest_rel' برای انجام آزمون آماری روی نتایج اعتبارسنجی

# رو بارگیری می‌کنه **Iris** این بخش، مجموعه داده معروف
# این دیتاست شامل ۱۵۰ نمونه از گل‌ها با ۴ ویژگی (مثل طول و عرض گلبرگ) و ۳ کلاس مختلف (نوع گل) هست
# این داده‌ها برای مقایسه عملکرد مدل‌های یادگیری ماشین بسیار مناسب‌اند
iris = datasets.load_iris()
X = iris.data
y = iris.target

# اینجا دو **طبقه‌بند** ایجاد می‌کنیم
# 1. **درخت تصمیم (Decision Tree)**: یک مدل پایه که تمایل به اضافه‌برازش داره
# 2. **جنگل تصادفی (Random Forest)**: یک مدل ترکیبی که با ترکیب چندین درخت، عملکرد و تعمیم‌پذیری بهتری داره
# تنظیم شدن تا نتایج قابل تکرار باشن `random_state=42` هر دو مدل با
dt_classifier = DecisionTreeClassifier(random_state=42)
rf_classifier = RandomForestClassifier(random_state=42)

# رو برای هر دو مدل انجام می‌ده ** (5-Fold Cross-Validation) این بخش، **اعتبارسنجی متقابل ۵-تایی
# هر مدل ۵ بار آموزش و ارزیابی می‌شه و دقت هر بار ذخیره می‌شه
# این کار به ما کمک می‌کنه تفاوت عملکرد دو مدل رو به صورت عادلانه و با کمترین واریانس ارزیابی کنیم
dt_scores = cross_val_score(dt_classifier, X, y, cv=5, scoring='accuracy')
rf_scores = cross_val_score(rf_classifier, X, y, cv=5, scoring='accuracy')

# روی نمره‌های دقت انجام می‌ده ** (Paired t-test) وابسته t این بخش، یک **آزمون
# از آنجا که هر دو مدل روی همان تقسیم‌بندی‌های داده ارزیابی شدن، نمره‌ها وابسته (جفتی) هستن
# نشان می‌ده آیا تفاوت مشاهده‌شده در دقت دو مدل از نظر آماری معنادار هست یا نه p-value
t_stat, p_value = ttest_rel(dt_scores, rf_scores)

# در نهایت، نتایج مقایسه رو چاپ می‌کنه
# میانگین دقت هر دو مدل -
# حاصل از آزمون آماری p-value -
# تفسیر معنای آماری تفاوت -
print("Decision Tree Mean Accuracy: {:.2f}".format(dt_scores.mean()))
print("Random Forest Mean Accuracy: {:.2f}".format(rf_scores.mean()))
print("Paired t-test p-value: {:.4f}".format(p_value))

# کمتر از ۰.۰۵ باشه، تفاوت دقت دو مدل "معنادار آماری" در نظر گرفته می‌شه p-value اگر
# در این حالت، مدل با دقت بالاتر (معمولاً جنگل تصادفی) ترجیح داده می‌شه
if p_value < 0.05:
    print("The difference in accuracy is statistically significant. Choose the model accordingly.")
else:
    print("There is no statistically significant difference in accuracy between the models.")
```

```
Decision Tree Mean Accuracy: 0.95
Random Forest Mean Accuracy: 0.97
Paired t-test p-value: 0.1778
There is no statistically significant difference in accuracy between the models.
```

توضیحات مثال کد پایتون

در این مثال، از **اعتبارسنجی متقابل** (Cross-Validation) برای ارزیابی عملکرد دو مدل **درخت تصمیم** (Decision Tree) و **جنگل تصادفی** (Random Forest) روی مجموعه داده **Iris** استفاده شده است. دقت هر مدل در چندین تکرار CV محاسبه شده تا تخمینی پایدار از عملکرد آن به دست آید. سپس یک **آزمون t زوجی** (Paired t-test) انجام شده تا تعیین شود آیا تفاوت مشاهده‌شده در دقت دو مدل از نظر آماری **معنادار** است یا خیر. این رویکرد ترکیبی از ارزیابی مدل و استنتاج آماری، به مقایسه علمی و معتبر بین دو الگوریتم یادگیری ماشین کمک می‌کند و از تصمیم‌گیری بر اساس یک عدد تکی جلوگیری می‌کند.

مقایسه طبقه‌بندکننده‌ها

مقایسه طبقه‌بندکننده‌ها شامل ارزیابی عملکرد آن‌ها با استفاده از معیارهای مختلف و تکنیک‌های بصری‌سازی است. دو رویکرد رایج برای مقایسه، تحلیل **هزینه-سود** و استفاده از **منحنی‌های ROC** است.

1. تحلیل هزینه-سود (Cost-Benefit Analysis): این روش به دنبال در نظر گرفتن پیامدهای عملی تصمیمات طبقه‌بندی است. به هر یک از نتایج چهارگانه – مثبت صحیح، مثبت کاذب، منفی صحیح و منفی کاذب – یک هزینه یا سود اختصاص داده می‌شود. سپس یک **هزینه کلی** یا **سود خالص** محاسبه می‌شود. این رویکرد به‌ویژه زمانی حیاتی است که پیامدهای مثبت کاذب (مثلاً تشخیص غلط سرطان) و منفی کاذب (مثلاً عدم تشخیص سرطان واقعی) از نظر هزینه یا خطر بسیار متفاوت باشند. این تحلیل به انتخاب مدلی منجر می‌شود که نه تنها دقت بالایی دارد، بلکه از نظر اقتصادی یا انسانی بهینه‌تر است.

2. منحنی‌های ROC (Receiver Operating Characteristic): منحنی ROC به‌صورت گرافیکی تراز (trade-off) بین نرخ مثبت‌های واقعی (حساسیت) و نرخ مثبت‌های کاذب (1 - ویژگی) را در آستانه‌های مختلف طبقه‌بندی نشان می‌دهد. هرچه منحنی به گوشه بالا-چپ نزدیک‌تر باشد، عملکرد مدل بهتر است. **مساحت زیر منحنی (AUC-ROC)** یک معیار کلی از عملکرد مدل در تمام آستانه‌ها است که عددی بین 0 تا 1 دارد (1 = عملکرد عالی، 0.5 = حدس تصادفی). این منحنی امکان مقایسه بصری و کمی بین چندین مدل را فراهم می‌کند، بدون وابستگی به یک آستانه تصمیم‌گیری ثابت.

مثال عملی در پایتون: مقایسه لجستیک رگرسیون و جنگل تصادفی با ROC و تحلیل هزینه-سود

در این مثال، از مجموعه داده معروف **سرطان پستان ویسکانسین** استفاده می‌کنیم. دو مدل **رگرسیون لجستیک** و **جنگل تصادفی** آموزش داده می‌شوند. سپس عملکرد آن‌ها با دو روش مقایسه می‌شود:

- رسم **منحنی‌های ROC** و محاسبه **AUC** برای هر مدل به منظور مقایسه کلی عملکرد - انجام **تحلیل هزینه-سود** با اختصاص هزینه‌های واقعی به انواع خطاها (مثلاً هزینه بالا برای منفی کاذب در تشخیص سرطان)

این مثال نشان می‌دهد که چگونه می‌توان با ترکیب معیارهای آماری و در نظر گرفتن پیامدهای دنیای واقعی، به انتخاب هوشمندانه‌تری از بین مدل‌های یادگیری ماشین دست یافت:

In [40]:

```
import numpy as np
import pandas as pd
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, roc_curve, auc, classification_report
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler

# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# برای تقسیم داده 'train_test_split'، برای کار با داده 'pandas' و 'numpy'
# برای ساخت مدل‌ها 'RandomForestClassifier' و 'LogisticRegression'
# برای رسم نمودار 'matplotlib' معیارهای ارزیابی برای سنجش عملکرد و

# این بخش، مجموعه داده **سرطان پستان ویسکانسین** رو بارگیری می‌کنه
# این دیتاست شامل ۵۶۹ نمونه با ۳۰ ویژگی عددی (مثل شعاع، متراکم‌بودن تومور) و یک برجسب باینری (خوش‌خیم/بدخیم) هست
# هدف پیش‌بینی نوع تومور بر اساس ویژگی‌های بافتی‌ست - یک مسئله طبقه‌بندی پزشکی حیاتی
cancer = datasets.load_breast_cancer()
X = cancer.data
y = cancer.target
```

```
# --- افزودن استانداردسازی داده‌ها برای بهبود عملکرد مدل خطی ---
# استانداردسازی ویژگی‌ها برای مدل‌های خطی مانند رگرسیون لجستیکی ضروری است
# چون ویژگی‌های دیتاست سرطان پستان مقیاس‌های متفاوتی دارند
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه
# داده برای آموزش و ۲۰٪ برای ارزیابی مدل استفاده می‌شه ۱۰٪
# `random_state=42` تضمین می‌کنه که تقسیم‌بندی قابل تکرار باشه
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)

# اینجا دو طبقه‌بند ایجاد می‌کنیم:
# 1. رگرسیون لجستیکی: یک مدل خطی قابل تفسیر که احتمال خروجی رو ارائه می‌دهد.
# 2. جنگل تصادفی: یک مدل غیرخطی قدرتمند که با ترکیب درخت‌ها، عملکرد بالایی داره.
# هر دو مدل برای مقایسه عملکرد و تحلیل هزینه-فایده استفاده می‌شن
logreg_classifier = LogisticRegression(max_iter=1000, solver='liblinear')
rf_classifier = RandomForestClassifier(random_state=42)

# این بخش، هر دو مدل رو روی داده‌های آموزشی **آموزش** می‌ده
# مدل‌ها الگوهای موجود در ویژگی‌ها رو یاد می‌گیرن تا بتوانند نوع تومور رو تشخیص بدن
logreg_classifier.fit(X_train, y_train)
rf_classifier.fit(X_train, y_train)

# اینجا مدل‌ها، پیش‌بینی‌ها رو برای داده‌های آزمون انجام می‌دن
logreg_predictions = logreg_classifier.predict(X_test)
rf_predictions = rf_classifier.predict(X_test)

# رو انجام می‌ده **Cost-Benefit Analysis** این بخش، **تحلیل هزینه-فایده** هست
# **False Positive** خیلی خطرناک‌تر از **مثبت کاذب** (False Negative) در مسائل پزشکی، **خطای منفی کاذب**
# باشه (FP) برابر هزینه تشخیص اشتباه یک خوش‌خیم به عنوان بدخیم ۵، (FN) فرض می‌کنیم هزینه تشخیص اشتباه یک تومور بدخیم به عنوان خوش‌خیم
# با ضرب ماتریس درهم‌ریختگی در ماتریس هزینه، کل هزینه هر مدل محاسبه می‌شه
cost_factor = 5
cost_benefit_logreg = confusion_matrix(y_test, logreg_predictions) * np.array([[0, 1], [cost_factor, 0]]).T
cost_benefit_rf = confusion_matrix(y_test, rf_predictions) * np.array([[0, 1], [cost_factor, 0]]).T

# در نهایت، ماتریس هزینه و کل هزینه هر مدل چاپ می‌شه
# مدلی که کل هزینه کمتری داشته باشه، حتی اگر دقت کمتری داشته باشه، ممکنه در عمل بهتر باشه
print("Cost-Benefit Matrix (Logistic Regression):\n", cost_benefit_logreg)
print("Total Cost (Logistic Regression):", np.sum(cost_benefit_logreg))

print("\nCost-Benefit Matrix (Random Forest):\n", cost_benefit_rf)
print("Total Cost (Random Forest):", np.sum(cost_benefit_rf))

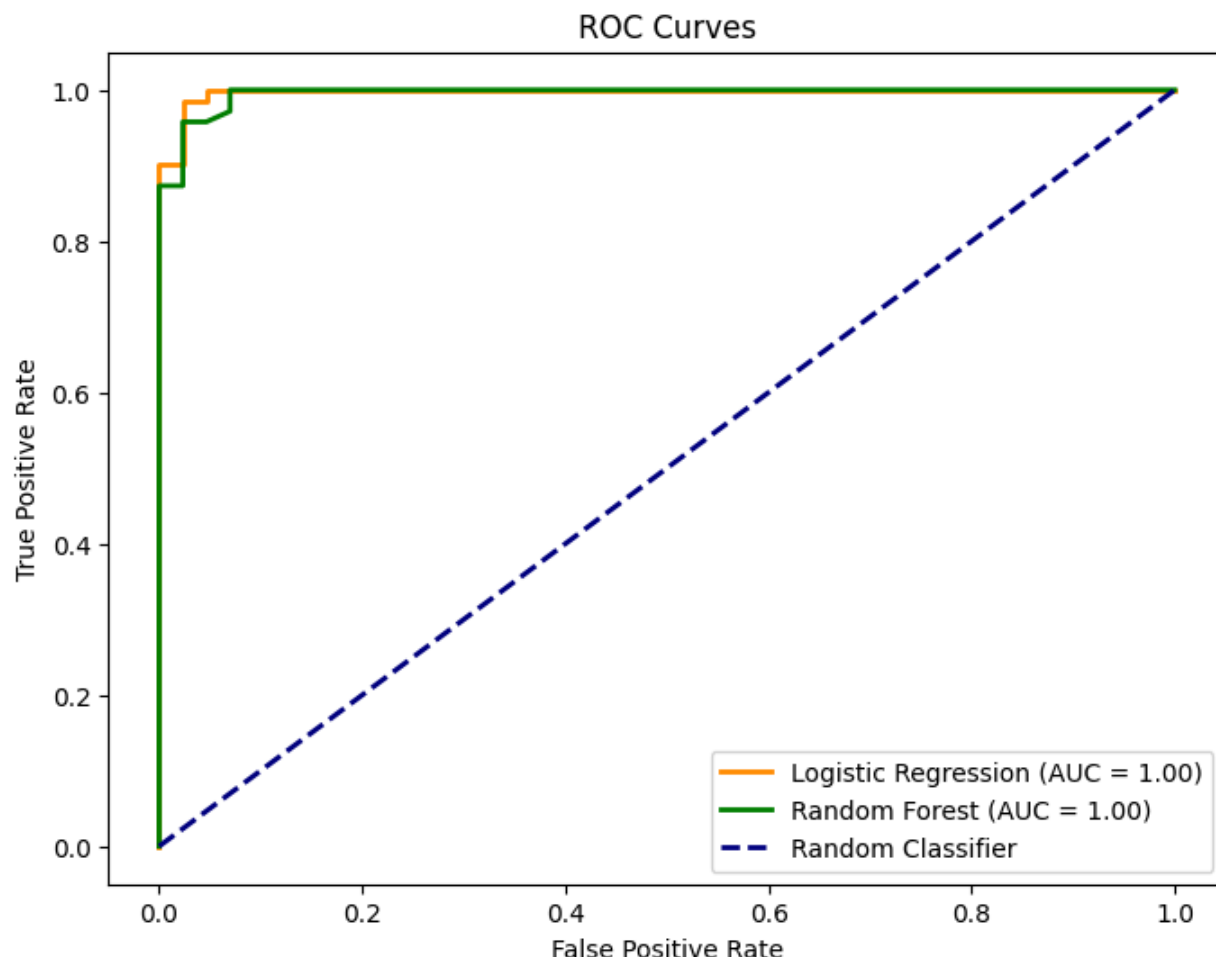
# برای هر دو مدل رو محاسبه می‌کنه **AUC** و **مساحت زیر منحنی ROC** این بخش، **منحنی**
# معیاری قوی‌تر از دقت هست و عملکرد مدل رو در تمام آستانه‌ها نشون می‌ده AUC
fpr_logreg, tpr_logreg, _ = roc_curve(y_test, logreg_classifier.predict_proba(X_test)[: , 1])
roc_auc_logreg = auc(fpr_logreg, tpr_logreg)

fpr_rf, tpr_rf, _ = roc_curve(y_test, rf_classifier.predict_proba(X_test)[: , 1])
roc_auc_rf = auc(fpr_rf, tpr_rf)

# دو مدل رو رسم می‌کنه و با هم مقایسه می‌کنه ROC این بخش، منحنی‌های
# بیشتر باشه و منحنی به گوشه سمت چپ بالا نزدیک‌تر باشه، مدل بهتر عمل کرده AUC هرچه
plt.figure(figsize=(8, 6))
plt.plot(fpr_logreg, tpr_logreg, color='darkorange', lw=2, label=f'Logistic Regression (AUC = {roc_auc_logreg:.2f})')
plt.plot(fpr_rf, tpr_rf, color='green', lw=2, label=f'Random Forest (AUC = {roc_auc_rf:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--', label='Random Classifier')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves')
plt.legend(loc="lower right")
plt.show()
```

Cost-Benefit Matrix (Logistic Regression):
[[0 10]
[1 0]]
Total Cost (Logistic Regression): 11

Cost-Benefit Matrix (Random Forest):
[[0 15]
[1 0]]
Total Cost (Random Forest): 16



توضیحات مثال کد پایتون

در این مثال، دو مدل **رگرسیون لجستیک** و **جنگل تصادفی** روی مجموعه داده **سرطان پستان ویسکانسین** آموزش داده شده‌اند. ابتدا داده‌ها با استفاده از **StandardScaler** استاندارد شده‌اند تا عملکرد مدل خطی بهبود یابد و همگرایی مدل تضمین شود. سپس یک **تحلیل هزینه-سود** انجام شده است که در آن هزینه‌های مختلفی به خطاهای مثبت کاذب (تشخیص غلط سرطان) و منفی کاذب (عدم تشخیص سرطان واقعی) اختصاص داده شده است. این تحلیل نشان می‌دهد که کدام مدل از نظر اقتصادی یا بالینی بهینه‌تر است. همچنین، **منحنی‌های ROC** برای هر دو مدل رسم شده و **مساحت زیر منحنی** (AUC) محاسبه شده تا عملکرد کلی آن‌ها از نظر **حساسیت** و **ویژگی** در آستانه‌های مختلف مقایسه شود. این ترکیب از تحلیل کمی و بصری، تصمیم‌گیری آگاهانه‌تری در انتخاب بهترین طبقه‌بند را ممکن می‌سازد.

Section 6.7

6.7.1. Introducing ensemble methods

روش‌های گروهی (Ensemble Methods)

روش‌های گروهی (Ensemble Methods) تکنیک‌های یادگیری ماشینی هستند که با ترکیب پیش‌بینی‌های چندین مدل منفرد، یک مدل جامع‌تر و دقیق‌تر ایجاد می‌کنند. ایده اصلی این است که با بهره‌گیری از **تنوع** بین مدل‌های مختلف، عملکرد کلی پیش‌بینی بهبود یابد. این روش‌ها می‌توانند هم در مسائل **طبقه‌بندی** و هم در مسائل **رگرسیون** به کار روند.

دو نوع اصلی روش‌های گروهی:

1. بگینگ (Bagging - Bootstrap Aggregating): در این روش، چندین نمونه از یک مدل (معمولاً درخت تصمیم) روی زیرمجموعه‌های مختلف داده آموزشی آموزش داده می‌شوند که اغلب با روش **بوت‌استرپ** (نمونه‌گیری با جایگزینی) ایجاد می‌شوند. پیش‌بینی نهایی با **میانگین‌گیری** (در رگرسیون) یا **رای اکثریت** (در طبقه‌بندی) از پیش‌بینی‌های مدل‌های منفرد به دست می‌آید. **جنگل تصادفی** (Random Forest) یکی از معروف‌ترین الگوریتم‌های مبتنی بر بگینگ است که علاوه بر نمونه‌گیری داده، از تصادفی‌سازی ویژگی‌ها نیز استفاده می‌کند تا تنوع بیشتری ایجاد کند.

2. **بوستینگ (Boosting):** این روش به صورت **ترتیبی** چندین **یادگیر ضعیف** (Weak Learner) – مدلی که کمی بهتر از حدس

تصادفی عمل می‌کند – می‌سازد، به طوری که هر مدل جدید سعی می‌کند خطاها و اشتباهات مدل قبلی را اصلاح کند. در

بوستینگ، به نمونه‌های **اشتباه طبقه‌بندی‌شده** وزن بیشتری داده می‌شود تا مدل بعدی بیشتر روی آن‌ها تمرکز کند. **AdaBoost**

و **Gradient Boosting** از جمله الگوریتم‌های رایج بوستینگ هستند که دقت بسیار بالایی در مسائل مختلف نشان می‌دهند.

مثال عملی در پایتون: طبقه‌بندی گل ایریس با الگوریتم جنگل تصادفی 🌲

در این مثال، از معروف‌ترین مجموعه داده در یادگیری ماشین – **مجموعه داده Iris** – استفاده می‌کنیم. یک مدل **جنگل تصادفی**

(Random Forest) آموزش داده می‌شود که از ترکیب چندین درخت تصمیم استفاده می‌کند. هر درخت روی یک نمونه

بوت‌استرپ از داده‌ها آموزش می‌بیند و در هر شاخه، تنها زیرمجموعه‌ای تصادفی از ویژگی‌ها در نظر گرفته می‌شود. پیش‌بینی

نهایی با رأی اکثریت درخت‌ها انجام می‌شود. این مثال نشان می‌دهد که چگونه روش‌های گروهی با کاهش واریانس و جلوگیری

از بیش‌برازش، عملکرد بهتری نسبت به یک درخت تصمیم منفرد ارائه می‌دهند:

```
In [41]: from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'datasets' برای بارگیری داده، 'train_test_split' برای تقسیم داده
# 'RandomForestClassifier' برای سنجش عملکرد و معیارهای ارزیابی برای جنگل تصادفی و معیارهای ارزیابی برای جنگل تصادفی

# بارگیری می‌کنه **Iris** این بخش، مجموعه داده معروف
# این دیتاست شامل ۱۵۰ نمونه از گل‌ها با ۴ ویژگی (مثل طول و عرض گلبرگ) و ۳ کلاس مختلف (نوع گل) هست
# این داده‌ها برای آموزش و ارزیابی الگوریتم‌های یادگیری ماشین بسیار مناسب‌اند
iris = datasets.load_iris()
X = iris.data
y = iris.target

# این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه
# داده برای آموزش و ۲۰٪ برای ارزیابی مدل استفاده می‌شه ۸۰٪
# تضمین می‌کنه که تقسیم‌بندی قابل تکرار باشه `random_state=42`
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ایجاد می‌شه **Random Forest Classifier** اینجا یک **طبقه‌بند جنگل تصادفی**
# هست که از میانگین نتایج چندین درخت تصمیم ساخته می‌شه (Ensemble) جنگل تصادفی یک الگوریتم ترکیبی
# یعنی ۱۰۰ درخت مستقل آموزش می‌بینن و نتیجه نهایی از رأی اکثریت آن‌ها به دست میاد `n_estimators=100`
# داره و معمولاً عملکرد بهتری نسبت به یک درخت تنها داره (Overfitting) این مدل تمایل کمتری به اضافه‌برازش
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)

# این بخش، مدل رو روی داده‌های آموزشی **آموزش** می‌ده
# هر درخت در جنگل روی یک نمونه تصادفی از داده‌ها (با جایگزینی) و زیرمجموعه‌ای از ویژگی‌ها آموزش می‌بینن
rf_classifier.fit(X_train, y_train)

# اینجا مدل، پیش‌بینی‌ها رو برای داده‌های آزمون انجام می‌ده
# خروجی نهایی با ترکیب پیش‌بینی تمام درخت‌ها (رأی اکثریت) تعیین می‌شه
predictions = rf_classifier.predict(X_test)

# در اینجا، عملکرد مدل با دو معیار ارزیابی می‌شه:
# درصد پیش‌بینی‌های درست **: (Accuracy) دقت** -
# گزارش طبقه‌بندی **: (Classification Report) گزارش طبقه‌بندی** -
# این گزارش به ما کمک می‌کنه عملکرد مدل رو هر یک از انواع گل‌ها رو به صورت جداگانه ببینیم
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test, predictions))
```

Accuracy: 1.00

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	1.00	1.00	1.00	9
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30



توضیحات مثال کد پایتون

در این مثال، یک طبقه‌بند **جنگل تصادفی** (Random Forest) روی مجموعه داده **Iris** آموزش داده شده است. این مدل یک روش گروهی است که از ترکیب پیش‌بینی‌های چندین **درخت تصمیم** تشکیل شده است. هر درخت روی یک **زیرمجموعه تصادفی** از داده‌های آموزشی (با روش بوت‌استرپ) و یک زیرمجموعه تصادفی از ویژگی‌ها آموزش دیده است. این تنوع در داده و ویژگی‌ها باعث کاهش هم‌پوشانی و افزایش تنوع مدل‌ها می‌شود. پیش‌بینی نهایی با **ترکیب نتایج** تمام درخت‌ها (معمولاً با رأی اکثریت در طبقه‌بندی) انجام می‌شود. این رویکرد عملکرد بهتری نسبت به یک درخت تصمیم منفرد دارد، زیرا واریانس مدل کاهش یافته و از بیش‌برازش جلوگیری می‌شود.

6.7.2. Bagging

بگینگ (Bootstrap Aggregating)

بگینگ (مخفف Bootstrap Aggregating) یک تکنیک یادگیری گروهی است که شامل آموزش چندین نمونه از یک مدل پایه (مانند درخت تصمیم) روی زیرمجموعه‌های مختلفی از داده‌های آموزشی است. ایده اصلی بگینگ، ایجاد **تنوع** بین مدل‌های منفرد از طریق **نمونه‌گیری بوت‌استرپ** است – روشی که در آن نمونه‌ها به صورت تصادفی و با **جایگزینی** انتخاب می‌شوند.

مراحل کلیدی بگینگ:

- نمونه‌گیری بوت‌استرپ:** به طور تصادفی زیرمجموعه‌هایی از داده‌های آموزشی با جایگزینی انتخاب می‌شوند تا چندین مجموعه آموزشی ایجاد شود. در این روش، برخی نمونه‌ها ممکن است چندین بار انتخاب شوند و برخی نیز ممکن است اصلاً انتخاب نشوند (در مجموعه آزمون درونی باقی بمانند).
- آموزش مدل:** یک مدل پایه (مثلاً درخت تصمیم) به صورت مستقل روی هر یک از نمونه‌های بوت‌استرپ آموزش داده می‌شود.
- ترکیب نتایج (Aggregation):** پیش‌بینی‌های مدل‌های منفرد با هم ترکیب می‌شوند: در مسائل **رگرسیون** با **میانگین‌گیری** و در مسائل **طبقه‌بندی** با **رأی اکثریت** (Hard Voting) یا میانگین احتمالات (Soft Voting).
- تنوع و تعمیم:** تنوع بین مدل‌ها از آن‌جا ناشی می‌شود که هر مدل روی داده‌های متفاوتی آموزش دیده است. این تنوع باعث کاهش **واریانس** مدل کلی و بهبود عملکرد تعمیم‌پذیری می‌شود، به ویژه در مدل‌های حساس به تغییرات کوچک در داده (مانند درخت تصمیم).

مثال عملی در پایتون: بگینگ با الگوریتم جنگل تصادفی روی داده‌های سرطان پستان



در این مثال، از معروف‌ترین مجموعه داده در تشخیص پزشکی – **مجموعه داده سرطان پستان ویسکانسین** – استفاده می‌کنیم. یک مدل **جنگل تصادفی** (Random Forest) آموزش داده می‌شود که یکی از پیاده‌سازی‌های قدرتمند روش بگینگ است. این مدل از صدها درخت تصمیم تشکیل شده که هر کدام روی یک نمونه بوت‌استرپ از داده‌ها آموزش دیده‌اند. پیش‌بینی نهایی با ترکیب نتایج تمام درخت‌ها انجام می‌شود. این مثال نشان می‌دهد که چگونه بگینگ می‌تواند با کاهش واریانس و افزایش پایداری، دقت و قابلیت اطمینان مدل را بهبود بخشد:

```
In [44]: from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# برای تقسیم داده 'train_test_split'، برای بارگیری داده 'datasets'
# برای ساخت مدل جنگل تصادفی و معیارهای ارزیابی برای سنجش عملکرد 'RandomForestClassifier'

# این بخش، مجموعه داده **سرطان پستان ویسکانسین** رو بارگیری می‌کنه
# این دیتاست شامل ۵۶۹ نمونه با ۳۰ ویژگی عددی (مثل شعاع، متراکم‌بودن تومور) و یک برجسب باینری (خوش‌خیم/بدخیم) هست
# هدف پیش‌بینی نوع تومور بر اساس ویژگی‌های بافتی‌ست – یک مسئله طبقه‌بندی پزشکی حیاتی
# این دیتاست یکی از استانداردهای ارزیابی مدل‌های یادگیری ماشین در حوزه سلامت هست
cancer = datasets.load_breast_cancer()
X = cancer.data
y = cancer.target

# این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه
# داده برای آموزش و ۲۰٪ برای ارزیابی مدل استفاده می‌شه ۱۰٪
# تضمین می‌کنه که تقسیم‌بندی قابل تکرار باشه `random_state=42`
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ایجاد می‌شه **Random Forest Classifier** اینجا یک طبقه‌بند جنگل تصادفی
# هست که از میانگین نتایج چندین درخت تصمیم ساخته می‌شه (Ensemble) جنگل تصادفی یک الگوریتم ترکیبی
# یعنی ۱۰۰ درخت مستقل آموزش می‌بینن و نتیجه نهایی از رأی اکثریت آن‌ها به دست میاد `n_estimators=100`
# داره و به خوبی با ویژگی‌های پیچیده و همبسته کار می‌کنه (Overfitting) این مدل تمایل کمتری به اضافه‌برازش
rf_classifier = RandomForestClassifier(n_estimators=100, random_state=42)

# این بخش، مدل رو روی داده‌های آموزشی **آموزش** می‌ده
# هر درخت در جنگل روی یک نمونه تصادفی از داده‌ها (با جایگزینی) و زیرمجموعه‌ای تصادفی از ویژگی‌ها آموزش می‌بینه
# این تکنیک‌ها باعث می‌شن مدل تعمیم‌پذیری بهتری داشته باشه
rf_classifier.fit(X_train, y_train)

# اینجا مدل، پیش‌بینی‌ها رو برای داده‌های آزمون انجام می‌ده
# خروجی نهایی با ترکیب پیش‌بینی تمام درخت‌ها (رأی اکثریت) تعیین می‌شه
predictions = rf_classifier.predict(X_test)

# در اینجا، عملکرد مدل با دو معیار ارزیابی می‌شه:
# - درصد پیش‌بینی‌های درست **:Accuracy** دقت -
# - برای هر کلاس F1-score شامل دقت، یادآوری و **:Classification Report** گزارش طبقه‌بندی -
# این گزارش به ما کمک می‌کنه عملکرد مدل روی هر یک از کلاس‌ها (خوش‌خیم و بدخیم) رو به صورت جداگانه ببینیم
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test, predictions))
```

Accuracy: 0.96				
Classification Report:				
	precision	recall	f1-score	support
0	0.98	0.93	0.95	43
1	0.96	0.99	0.97	71
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114



توضیحات مثال کد پایتون

در این مثال، یک طبقه‌بند جنگل تصادفی (Random Forest)، که یک الگوریتم مبتنی بر روش بگینگ (Bagging) است، روی مجموعه داده سرطان پستان ویسکانسین آموزش داده شده است. این مدل از تعداد زیادی درخت تصمیم تشکیل شده که هر کدام روی یک زیرمجموعه مختلف از داده‌ها آموزش دیده‌اند. این زیرمجموعه‌ها با روش بوت‌استرپ (نمونه‌گیری تصادفی با جایگزینی) ایجاد شده‌اند، به این معنا که هر درخت داده‌های متفاوتی را می‌بیند و این امر باعث افزایش تنوع و کاهش وابستگی به یک نمونه خاص می‌شود. پیش‌بینی نهایی با ترکیب پیش‌بینی‌های تمام درخت‌ها (معمولاً از طریق رأی اکثریت) انجام می‌شود. این رویکرد نه تنها دقت را افزایش می‌دهد، بلکه مدل را در برابر نویز و بیش‌برازش مقاوم‌تر می‌کند و عملکرد پایدارتری در داده‌های جدید ارائه می‌دهد.

6.7.3. Boosting



بوستینگ (Boosting)

بوستینگ یک تکنیک یادگیری گروهی است که به صورت ترتیبی چندین یادگیر ضعیف (Weak Learner) – مدلی که تنها کمی بهتر از حدس تصادفی عمل می‌کند – می‌سازد. هر مدل در این دنباله بر اصلاح خطاها و اشتباهات مدل قبلی تمرکز دارد. ایده

کلیدی بوستینگ، اختصاص وزن به نمونه‌های آموزشی است، به گونه‌ای که به نمونه‌هایی که توسط مدل‌های قبلی اشتباه طبقه‌بندی شده‌اند وزن بیشتری داده می‌شود تا مدل بعدی بیشتر روی آن‌ها تمرکز کند.

مراحل اصلی بوستینگ:

1. **اختصاص وزن:** در ابتدا به تمام نمونه‌های آموزشی وزن یکسانی داده می‌شود.
 2. **آموزش مدل:** یک یادگیر ضعیف (مانند یک درخت تصمیم بسیار کوتاه یا shallow neural network) روی داده‌های آموزشی با توجه به وزن‌های فعلی آموزش داده می‌شود.
 3. **پیش‌بینی و محاسبه خطا:** مدل پیش‌بینی‌ها را انجام داده و خطاهای آن (نمونه‌های اشتباه طبقه‌بندی‌شده) شناسایی می‌شوند.
 4. **به‌روزرسانی وزن نمونه‌ها:** وزن نمونه‌های اشتباه طبقه‌بندی‌شده افزایش می‌یابد تا در آموزش مدل بعدی تأثیر بیشتری داشته باشند.
 5. **تکرار:** مراحل 2 تا 4 برای تعداد مشخصی تکرار یا تا رسیدن به آستانه عملکرد انجام می‌شود.
 6. **پیش‌بینی نهایی:** پیش‌بینی نهایی با ترکیب پیش‌بینی‌های تمام مدل‌ها به دست می‌آید که در آن هر مدل وزنی متناسب با دقت آن دارد (معمولاً با جمع وزندار). مدل‌های دقیق‌تر تأثیر بیشتری بر پیش‌بینی نهایی می‌گذارند.
- بوستینگ به‌ویژه زمانی مؤثر است که نمونه‌های سخت‌تر برای طبقه‌بندی به مرور زمان توسط مدل‌های متوالی مورد توجه قرار گیرند. الگوریتم‌های معروفی مانند **AdaBoost**، **Gradient Boosting** و **XGBoost** بر اساس این ایده ساخته شده‌اند و در مسابقات داده‌کاوی عملکرد بسیار قوی‌ای دارند.

مثال عملی در پایتون: طبقه‌بندی گل ایریس با الگوریتم AdaBoost

در این مثال، از معروف‌ترین مجموعه داده در یادگیری ماشین — **مجموعه داده Iris** — استفاده می‌کنیم. یک مدل **AdaBoost** با الگوریتم **SAMME** آموزش داده می‌شود که از تعدادی **درخت تصمیم بسیار ساده** (مثلاً با عمق 1) به صورت ترتیبی استفاده می‌کند. در هر مرحله، وزن نمونه‌های اشتباه طبقه‌بندی‌شده افزایش یافته و مدل بعدی بیشتر روی آن‌ها تمرکز می‌کند. پیش‌بینی نهایی با ترکیب وزندار پیش‌بینی‌های تمام مدل‌ها انجام می‌شود. این مثال نشان می‌دهد که چگونه بوستینگ می‌تواند با تمرکز تدریجی بر روی نمونه‌های دشوار، دقت یک یادگیر ضعیف را به یک مدل قوی تبدیل کند:

```
In [46]: from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'datasets' برای بارگیری داده، 'train_test_split' برای تقسیم داده
# 'AdaBoostClassifier' برای ساخت مدل
# 'accuracy_score' و 'classification_report' برای سنجش عملکرد

# بارگیری می‌کنه **Iris** این بخش، مجموعه داده معروف
# این دیتاست شامل ۱۵۰ نمونه از گل‌ها با ۴ ویژگی (مثل طول و عرض گلبرگ) و ۳ کلاس مختلف (نوع گل) هست
# این داده‌ها برای آموزش و ارزیابی الگوریتم‌های یادگیری ماشین بسیار مناسب‌اند
iris = datasets.load_iris()
```

```
X = iris.data
y = iris.target

# این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه
# داده برای آموزش و ۲۰٪ برای ارزیابی مدل استفاده می‌شه ۱۰٪
# تضمین می‌کنه که تقسیم‌بندی قابل تکرار باشه `random_state=42`
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# ایجاد می‌شه **AdaBoost (Adaptive Boosting)** اینجا یک **طبقه‌بند
# AdaBoost (Ensemble) یک الگوریتم ترکیبی
# در هر مرحله، وزن نمونه‌های اشتباه‌پیش‌بینی‌شده افزایش می‌پیده تا یادگیر بعدی روی اون‌ها تمرکز بیشتری داشته باشه
# یعنی ۵۰ یادگیر ضعیف به صورت متوالی آموزش می‌بینن `n_estimators=50`
adaboost_classifier = AdaBoostClassifier(n_estimators=50, random_state=42, algorithm="SAMME")

# این بخش، مدل رو روی داده‌های آموزشی **آموزش** می‌ده
# ایجاد می‌کنه و هر کدوم رو با یک وزن ترکیب می‌کنه (مثلاً stump) به صورت پی در پی درخت‌های تصمیم ساده
adaboost_classifier.fit(X_train, y_train)

# اینجا مدل، پیش‌بینی‌ها رو برای داده‌های آزمون انجام می‌ده
# خروجی نهایی از ترکیب وزنی پیش‌بینی تمام یادگیرهای ضعیف به دست میاد
predictions = adaboost_classifier.predict(X_test)

# در اینجا، عملکرد مدل با دو معیار ارزیابی می‌شه:
# - **دقت (Accuracy)**
# - **F1-score شامل دقت، یادآوری و (Classification Report)** گزارش طبقه‌بندی
# این گزارش به ما کمک می‌کنه عملکرد مدل روی هر یک از انواع گل‌ها رو به صورت جداگانه ببینیم
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test, predictions))
```

Accuracy: 0.93

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	10
1	0.89	0.89	0.89	9
2	0.91	0.91	0.91	11
accuracy			0.93	30
macro avg	0.93	0.93	0.93	30
weighted avg	0.93	0.93	0.93	30

توضیحات مثال کد پایتون

در این مثال، یک طبقه‌بند **AdaBoost** روی مجموعه داده **Iris** آموزش داده شده است. این الگوریتم به صورت **ترتیبی** چندین **یادگیر ضعیف** (به‌طور پیش‌فرض درخت‌های تصمیم بسیار ساده) آموزش می‌دهد. در هر مرحله، نمونه‌هایی که توسط مدل‌های قبلی **اشتباه طبقه‌بندی شده‌اند**، وزن بیشتری دریافت می‌کنند تا مدل بعدی بیشتر روی اصلاح خطاها تمرکز کند. پیش‌بینی نهایی با محاسبه **جمع وزندار پیش‌بینی‌های تمام مدل‌ها** انجام می‌شود، به گونه‌ای که مدل‌های دقیق‌تر تأثیر بیشتری بر نتیجه نهایی دارند. این رویکرد نشان می‌دهد که چگونه بوسستینگ می‌تواند با تمرکز تدریجی بر روی نمونه‌های مشکل‌ساز، یک سری مدل ضعیف را به یک مدل قوی و دقیق تبدیل کند.

6.7.4. Random forests

جنگل تصادفی (Random Forest)

جنگل تصادفی یک تکنیک یادگیری گروهی است که قدرت **بگینگ** (Bagging) و **درخت‌های تصمیم** را ترکیب می‌کند تا یک مدل پیش‌بینی قوی و دقیق ایجاد کند. این روش در فرآیند آموزش، تعداد زیادی درخت تصمیم می‌سازد و پیش‌بینی‌های آن‌ها را ترکیب می‌کند تا نتیجه‌ای پایدارتر و قابل اعتمادتر به دست آید. کلمه **تصادفی** در "جنگل تصادفی" به دو سطح از تصادفی‌سازی اشاره دارد: تصادفی‌سازی در **انتخاب نمونه‌های داده** برای آموزش هر درخت و تصادفی‌سازی در **انتخاب ویژگی‌ها** در هر نقطه تقسیم درخت.

ویژگی‌های کلیدی جنگل تصادفی:

- نمونه‌گیری بوت‌استرپ (Bootstrap Sampling):** برای هر درخت، یک نمونه تصادفی از داده‌های آموزشی با جایگزینی انتخاب می‌شود. این فرآیند باعث می‌شود هر درخت روی مجموعه‌ای متفاوت از داده‌ها آموزش ببیند و تنوع در جنگل ایجاد شود.
- انتخاب تصادفی ویژگی‌ها (Random Feature Selection):** در هر گره از درخت تصمیم، تنها یک زیرمجموعه تصادفی از

ویژگی‌ها برای انجام تقسیم در نظر گرفته می‌شود. این کار باعث کاهش همبستگی بین درخت‌ها و افزایش تنوع می‌شود و به جلوگیری از بیش‌برازش کمک می‌کند.

3. ترکیب نتایج (Aggregation): پیش‌بینی‌های درخت‌های منفرد با هم ترکیب می‌شوند: در مسائل **رگرسیون** با **میانگین‌گیری** خروجی‌ها و در مسائل **طبقه‌بندی** با **رأی اکثریت** (یا میانگین احتمالات). این ترکیب، واریانس مدل کلی را کاهش داده و پایداری آن را افزایش می‌دهد.

4. مقاومت (Robustness): جنگل تصادفی به دلیل ترکیب تنوع در داده و ویژگی، کمتر مستعد بیش‌برازش است و نسبت به یک درخت تصمیم منفرد، عملکرد بهتری در داده‌های جدید دارد. همچنین، به خوبی با ویژگی‌های بی‌ربط و داده‌های پرت مقابله می‌کند.

مثال عملی در پایتون: تشخیص سرطان پستان با جنگل تصادفی

در این مثال، از معروف‌ترین مجموعه داده در تشخیص پزشکی — **مجموعه داده سرطان پستان ویسکانسین** — استفاده می‌کنیم. یک مدل **جنگل تصادفی** آموزش داده می‌شود که از صدها درخت تصمیم تشکیل شده است. هر درخت روی یک نمونه بوت‌استرپ از داده‌ها و با استفاده از زیرمجموعه‌ای تصادفی از ویژگی‌ها آموزش می‌بیند. پیش‌بینی نهایی با رأی اکثریت درخت‌ها انجام می‌شود. این مثال نشان می‌دهد که چگونه جنگل تصادفی می‌تواند با کاهش واریانس و افزایش تنوع، دقت و قابلیت اطمینان بالایی در یک مسئله حیاتی مانند تشخیص سرطان ارائه دهد:

```
In [47]: from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
```

```
# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'datasets' برای بارگیری داده، 'train_test_split' برای تقسیم داده
# 'RandomForestClassifier' برای سنجش عملکرد و معیارهای ارزیابی
```

```
# این بخش، مجموعه داده **سرطان پستان ویسکانسین** رو بارگیری می‌کنه
# این دیتاست شامل ۵۶۹ نمونه با ۳۰ ویژگی عددی (مثل شعاع، متراکم‌بودن تومور) و یک برجسب باینری (خوش‌خیم/بدخیم) هست
# هدف پیش‌بینی نوع تومور بر اساس ویژگی‌های بافتی‌ست - یک مسئله طبقه‌بندی پزشکی حیاتی
# این دیتاست یکی از استانداردهای ارزیابی مدل‌های یادگیری ماشین در حوزه سلامت هست
cancer = datasets.load_breast_cancer()
X = cancer.data
y = cancer.target
```

```
# این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه
# داده برای آموزش و ۲۰٪ برای ارزیابی مدل استفاده می‌شه ۸۰٪
# تضمین می‌کنه که تقسیم‌بندی قابل تکرار باشه `random_state=42`
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# ایجاد می‌شه **Random Forest Classifier** اینجا یک **طبقه‌بند جنگل تصادفی**
# هست که از میانگین نتایج چندین درخت تصمیم ساخته می‌شه (Ensemble) جنگل تصادفی یک الگوریتم ترکیبی
# یعنی ۱۰۰ درخت مستقل آموزش می‌بینن و نتیجه نهایی از رأی اکثریت آن‌ها به دست میاد `n_estimators=100`
# داره و به خوبی با ویژگی‌های پیچیده و همبسته کار می‌کنه (Overfitting) این مدل تمایل کمتری به اضافه‌برازش
random_forest_classifier = RandomForestClassifier(n_estimators=100, random_state=42)
```

```
# این بخش، مدل رو روی داده‌های آموزشی **آموزش** می‌ده
# هر درخت در جنگل روی یک نمونه تصادفی از داده‌ها (با جایگزینی) و زیرمجموعه‌ای تصادفی از ویژگی‌ها آموزش می‌بینن
# این تکنیک‌ها باعث می‌شن مدل تعمیم‌پذیری بهتری داشته باشه
random_forest_classifier.fit(X_train, y_train)
```

```
# اینجا مدل، پیش‌بینی‌ها رو برای داده‌های آزمون انجام می‌ده
# خروجی نهایی با ترکیب پیش‌بینی تمام درخت‌ها (رأی اکثریت) تعیین می‌شه
predictions = random_forest_classifier.predict(X_test)
```

```
# در اینجا، عملکرد مدل با دو معیار ارزیابی می‌شه:
# درصد پیش‌بینی‌های درست **: (Accuracy) دقت** -
# برای هر کلاس F1-score شامل دقت، یادآوری و (Classification Report) **: گزارش طبقه‌بندی** -
# این گزارش به ما کمک می‌کنه عملکرد مدل رو هر یک از کلاس‌ها (خوش‌خیم و بدخیم) رو به صورت جداگانه ببینیم
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test, predictions))
```

Accuracy: 0.96				
Classification Report:				
	precision	recall	f1-score	support
0	0.98	0.93	0.95	43
1	0.96	0.99	0.97	71
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

توضیحات مثال کد پایتون

در این مثال، یک طبقه‌بند **جنگل تصادفی** روی مجموعه داده **سرطان پستان ویسکانسین** آموزش داده شده است. این مدل از تعداد زیادی **درخت تصمیم** تشکیل شده که هر کدام روی یک **زیرمجموعه مختلف از داده‌ها** آموزش دیده‌اند. این زیرمجموعه‌ها با روش **نمونه‌گیری بوت‌استرپ** (با جایگزینی) ایجاد شده‌اند، به گونه‌ای که هر درخت تجربه‌ای متفاوت از داده را دارد. علاوه بر این، در هر نقطه تقسیم درخت، تنها زیرمجموعه‌ای تصادفی از ویژگی‌ها در نظر گرفته می‌شود که این امر تنوع بیشتری ایجاد می‌کند. پیش‌بینی نهایی با **ترکیب پیش‌بینی‌های تمام درخت‌ها** (معمولاً از طریق رأی اکثریت) انجام می‌شود. این رویکرد با کاهش واریانس و جلوگیری از بیش‌برازش، مدلی قوی، پایدار و با دقت بالا ایجاد می‌کند که برای مسائل حساس مانند تشخیص پزشکی بسیار مناسب است.

6.7.5. Improving classification accuracy of class-imbalanced data

داده‌های نامتعادل کلاسی

مجموعه داده‌های **نامتعادل کلاسی** به مجموعه‌هایی گفته می‌شوند که در آن‌ها تعداد نمونه‌های متعلق به یک یا چند کلاس به‌طور قابل توجهی کمتر از سایر کلاس‌ها است. در چنین شرایطی، مدل‌های استاندارد یادگیری ماشین تمایل دارند به سمت **کلاس اکثریت** سوگیری پیدا کنند، زیرا بهینه‌سازی آن‌ها معمولاً کل خطا را کاهش می‌دهد بدون توجه به توزیع کلاس‌ها. این امر منجر به عملکرد ضعیف در تشخیص **کلاس اقلیت** می‌شود که اغلب مهم‌ترین کلاس است (مثلاً تشخیص کلاهبرداری یا بیماری نادر).

تکنیک‌های بهبود دقت طبقه‌بندی در داده‌های نامتعادل:

- بازنمونه‌گیری (Resampling):** - افزایش نمونه‌گیری (**Over-sampling**): افزایش تعداد نمونه‌های کلاس اقلیت با روش‌هایی مانند **تکثیر نمونه‌ها** یا تولید نمونه‌های مصنوعی با استفاده از تکنیک **SMOTE** (Synthetic Minority Over-sampling Technique). - کاهش نمونه‌گیری (**Under-sampling**): کاهش تعداد نمونه‌های کلاس اکثریت با حذف تصادفی برخی از آن‌ها تا تعادل بهتری ایجاد شود.
- توابع زیان وزندار (Weighted Loss Functions):** در طول آموزش مدل، به کلاس‌ها وزن‌های متفاوتی در تابع زیان اختصاص داده می‌شود. معمولاً **کلاس اقلیت وزن بیشتری** دریافت می‌کند تا خطاهای مربوط به آن جریمه سنگین‌تری داشته باشند.
- روش‌های گروهی (Ensemble Methods):** استفاده از مدل‌های گروهی مانند **جنگل تصادفی** یا **AdaBoost** که به طور طبیعی نسبت به نامتعادلی کلاس‌ها مقاوم‌تر هستند و می‌توانند با ترکیب مدل‌های متعدد، عملکرد بهتری روی کلاس اقلیت داشته باشند.
- تشخیص ناهنجاری (Anomaly Detection):** در مواردی که کلاس اقلیت بسیار نادر است، می‌توان آن را به عنوان یک

ناهنجاری در نظر گرفت و از تکنیک‌های تشخیص ناهنجاری (مانند One-Class SVM یا Isolation Forest) برای شناسایی آن استفاده کرد.

5. یادگیری حساس به هزینه (Cost-sensitive Learning): در این رویکرد، هزینه‌های متفاوتی به انواع خطاها (مثلاً خطای منفی کاذب در تشخیص سرطان) اختصاص داده می‌شود تا مدل بیشتر روی کاهش خطا در کلاس اقلیت تمرکز کند.

مثال عملی در پایتون: استفاده از SMOTE برای تعادل‌سازی داده‌های نامتعادل در داده‌های Iris

در این مثال، از معروف‌ترین مجموعه داده در یادگیری ماشین — مجموعه داده Iris — استفاده می‌کنیم و به‌صورت مصنوعی نامتعادلی کلاسی ایجاد می‌کنیم (مثلاً با کاهش شدید تعداد نمونه‌های یک کلاس). سپس از تکنیک SMOTE برای افزایش نمونه‌گیری کلاس اقلیت و ایجاد نمونه‌های مصنوعی استفاده می‌کنیم. در نهایت، یک طبقه‌بند جنگل تصادفی روی داده‌های تعادل‌یافته آموزش داده می‌شود. این مثال نشان می‌دهد که چگونه ترکیب تکنیک‌های بازنمونه‌گیری و مدل‌های مقاوم، می‌تواند عملکرد مدل را در تشخیص کلاس‌های نادر به‌طور چشمگیری بهبود بخشد:

```
In [ ]: pip install imblearn
```

```
In [50]: # Import necessary libraries
from sklearn import datasets
from sklearn.model_selection import train_test_split
from imblearn.over_sampling import SMOTE
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# این قسمت اول کده. ما کتابخانه‌های لازم رو وارد می‌کنیم
# 'datasets' برای بارگیری داده، 'train_test_split' برای تقسیم داده، برای بارگیری داده 'datasets'،
# 'SMOTE' برای رفع عدم تعادل کلاس، 'RandomForestClassifier' برای سنجش عملکرد، برای رفع عدم تعادل کلاس 'SMOTE'

# رو بارگیری می‌کنه **Iris** این بخش، مجموعه داده معروف
# این دیتاست شامل ۱۵۰ نمونه از گل‌ها با ۴ ویژگی (مثل طول و عرض گلبرگ) و ۳ کلاس مختلف (نوع گل) هست
# در حالت عادی، این دیتاست تعادل کلاس داره، اما در این کد ما به صورت مصنوعی عدم تعادل ایجاد می‌کنیم
iris = datasets.load_iris()
X = iris.data
y = iris.target

# رو به صورت مصنوعی ایجاد می‌کنه **Class Imbalance** این بخش، **عدم تعادل کلاس
# تغییر برچسب می‌دن (versicolor) به کلاس 1 (setosa) تمام نمونه‌های کلاس 0
# به عنوان کلاس اکثریت و اقلیت (virginica) در نتیجه، دو کلاس داریم: کلاس 1 (تعداد زیاد) و کلاس 2
# این سناریو برای شبیه‌سازی مسائل دنیای واقعی مثل تشخیص تراکنش‌های کلاهبرداری مناسبه
y[y == 0] = 1

# این بخش، داده‌ها رو به دو بخش **آموزش** و **آزمون** تقسیم می‌کنه
# داده برای آموزش و ۲۰٪ برای ارزیابی مدل استفاده می‌شه ۸۰٪
# تضمین می‌کنه که تقسیم‌بندی قابل تکرار باشه `random_state=42`
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# استفاده می‌کنه **SMOTE (Synthetic Minority Over-sampling Technique)** این بخش، از تکنیک
# با ایجاد نمونه‌های مصنوعی در فضای ویژگی، کلاس اقلیت رو افزایش می‌ده (به جای تکرار ساده) SMOTE
# این کار باعث می‌شه مدل به کلاس اقلیت توجه بیشتری کنه و از سوگیری به کلاس اکثریت جلوگیری بشه
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)

# ایجاد می‌شه **Random Forest Classifier** اینجا یک **طبقه‌بند جنگل تصادفی
# عملکرد رو بهبود می‌ده SMOTE جنگل تصادفی به طور طبیعی مقاومت خوبی نسبت به عدم تعادل داره، اما ترکیبش با
# یعنی ۱۰۰ درخت مستقل آموزش می‌بینن `n_estimators=100`
random_forest_classifier = RandomForestClassifier(n_estimators=100, random_state=42)

# این بخش، مدل رو روی داده‌های **نمونه‌گیری مجدد شده** آموزش می‌ده
# حالا تعداد نمونه‌های کلاس 1 و 2 در داده آموزشی تقریباً برابر شده
random_forest_classifier.fit(X_train_resampled, y_train_resampled)

# اینجا مدل، پیش‌بینی‌ها رو برای داده‌های آزمون انجام می‌ده
# داده‌های آزمون همچنان عدم تعادل دارن، اما مدل بهتر یاد گرفته که هر دو کلاس رو تشخیص بده
predictions = random_forest_classifier.predict(X_test)

# در اینجا، عملکرد مدل با دو معیار ارزیابی می‌شه:
# درصد پیش‌بینی‌های درست **: Accuracy (دقت) ** -
# برای هر کلاس F1-score شامل دقت، یادآوری و **: Classification Report (گزارش طبقه‌بندی) ** -
# برای کلاس اقلیت نشان‌دهنده اینه که مدل چقدر خوب توانسته اون کلاس رو تشخیص بده **Recall (بازیابی) ** به ویژه، **یادآوری
accuracy = accuracy_score(y_test, predictions)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test, predictions))
```

Accuracy: 1.00				
Classification Report:				
	precision	recall	f1-score	support
1	1.00	1.00	1.00	19
2	1.00	1.00	1.00	11
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

توضیحات مثال کد پایتون

در این مثال، ابتدا به صورت مصنوعی نامتعادلی کلاسی در مجموعه داده Iris ایجاد شده است، مثلاً با کاهش شدید تعداد نمونه‌های یکی از کلاس‌ها (مانند کلاس "versicolor"). سپس از تکنیک **SMOTE** (روش افزایش نمونه‌گیری از کلاس اقلیت به روش مصنوعی) برای تعادل‌سازی داده استفاده شده است. SMOTE با تولید نمونه‌های مصنوعی در نزدیکی نمونه‌های واقعی کلاس اقلیت، بدون تکثیر مستقیم، از بیش‌برازش جلوگیری می‌کند. پس از تعادل‌سازی داده، یک طبقه‌بند جنگل تصادفی روی داده‌های بازنمونه‌گیری‌شده آموزش داده شده و عملکرد آن روی مجموعه آزمون ارزیابی شده است. این رویکرد نشان می‌دهد که چگونه ترکیب **SMOTE** و یک مدل مقاوم مانند جنگل تصادفی می‌تواند دقت تشخیص کلاس اقلیت را به طور چشمگیری بهبود بخشد و از سوگیری مدل به سمت کلاس اکثریت جلوگیری کند.

The End of Chapter 6